MODULE-1

NUMBER SYSTEMS

Short Question and Answers

1) Define Number System. List out different types of number systems?

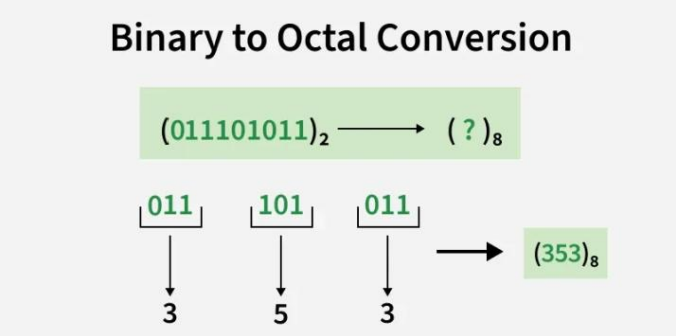
Ans: A number system is a method of representing numbers using a set of digits or symbols according to certain rules. It is used to store, process, and communicate data in digital devices.

- In digital electronics, there are four types of digital number systems are mainly used:
  - Binary Number System
  - Octal Number System
  - Decimal Number System
  - Hexadecimal Number System.

2) Convert $(011101011)_2$ into an octal number.

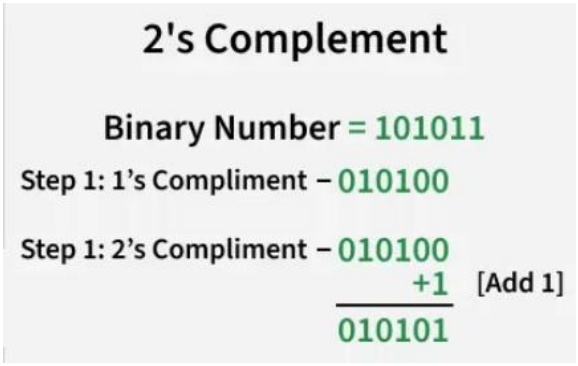
Ans: Step 1: Divide the binary number into groups of three digits starting from right to left i.e. from LSB to MSB.

Step 2: Convert these groups into equivalent octal digits.



3) What is 2's Complement? Give one example.

Ans: To find the 2's complement, calculate the 1's complement first, then add **1** to the least significant bit (the far right). This is the standard method computers use to represent signed integers.



4) Define De Morgan's Law.

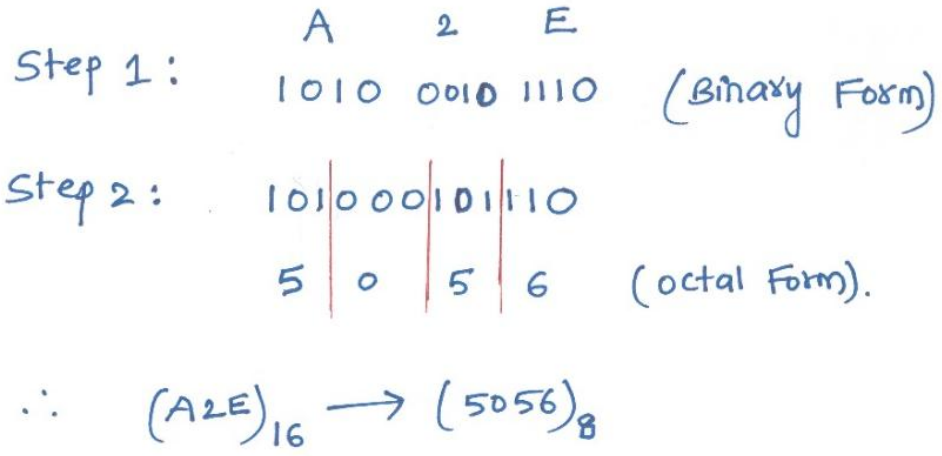
Ans: This law state that the complement of the AND operation of two variables is equal to the OR operation of their complements, and the complement of the OR operation of two variables is equal to the AND operation of their complements. Mathematically, the laws are expressed as:

$$\overline{A.B} = \overline{A} + \overline{B}$$
$$\overline{A + B} = \overline{A}.\overline{B}$$

5) Convert $(A2E)_{16}$ to Octal number.

Ans: **Step 1:** To convert Hexadecimal to Octal, first we need to convert the Hexadecimal number to Binary.

Step 2: Now to convert the binary number to an Octal number, divide the binary digits into groups of three digits starting from right to left i.e. from LSB to MSB.



6) Convert $(76543)_8$ to Hexadecimal.

Ans: **Step 1:** To convert Octal to Hexadecimal, first we need to convert the Octal to number to Binary.

Step 2: Now to convert the binary number to Hexadecimal number by divide the binary digits into groups of four digits starting from right to left i.e. from LSB to MSB.

Step 1 : 7 6 5 4 3
111 110 101 100 011 (octal Form)

Step 2: 0111 1101 0110 0011
7 D 6 3 (Hexadecimal Form)

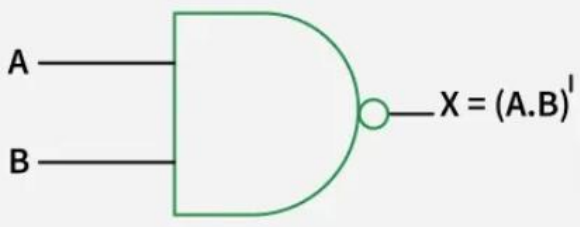
$\therefore (76543)_8 \longrightarrow (7D63)_{16}$

7) Explain about NAND gate With symbol and truth table.

Ans: The NAND gate is the combination of two basic logic gates AND gate and the NOT gate connected in series.

- The output of the NAND gate will give result high (1) when either of its input is high (1) or both of its input are low (0). In simple, it performs the inverted operation of AND gate.
- The Boolean Expression of NAND Gate is as follows:

$$\mathbf{X = \overline{A.B}} \quad \text{or} \quad \mathbf{(A.B)'}$$



Truth Table

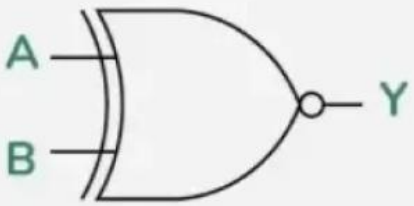
Input A	Input B	X = (A.B)'
0	0	1
0	1	1
1	0	1
1	1	0

8) Explain about XNOR gate with symbol and truth table.

Ans: The XNOR is the combination of XOR gate and NOT gate.

- The output of the XNOR gate is high(1) when both the inputs are high (1) or low(0). In other words, the output of the XNOR gate is high(1) when both the inputs are the same.
- The following is the Boolean expression of the XNOR gate,

$$Y = (A \oplus B)' \quad \text{or} \quad \overline{(A \oplus B)}$$



Truth Table

Input A	Input B	Output
0	0	1
0	1	0
1	0	0
1	1	1

9) Express the Boolean function F = A + B'C in a sum of minterms.

Given that, $F(A,B,C) = A + B'C$

$= A(B+B')(C+C') + B'C(A+A')$

$= ABC + ABC' + AB'C + AB'C' + AB'C + A'B'C$

$= ABC + ABC' + AB'C + AB'C' + A'B'C$

$= m_7 + m_6 + m_5 + m_4 + m_1$

$\therefore F(A,B,C) = \Sigma(1,4,5,6,7)$

10) Express F = xy + x'z in a product of maxterms form.

Ans:

$F = xy + x'z$

$F = (xy + x')(xy + z)$

$F = (x + x')(y + x')(x + z)(y + z)$ remember $x + x' = 1$

$F = (y + x')(x + z)(y + z)$

$F = (x' + y + zz')(x + yy' + z)(xx' + y + z)$

$F = (x' + y + z)(x' + y + z')(x + y + z)(x + y' + z)(x + y + z)(x' + y + z)$

$F = (x' + y + z)(x' + y + z')(x + y + z)(x + y' + z)$

$F = M_4 M_5 M_0 M_2$

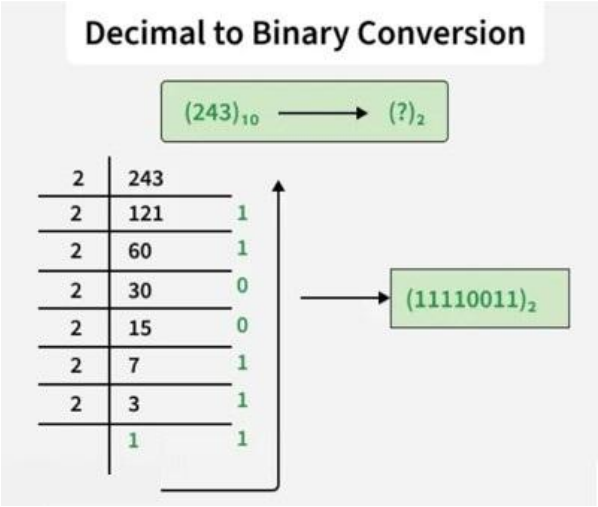
$F(x, y, z) = \Pi(0, 2, 4, 5)$

Long Question and Answers

1) Convert $(243)_{10}$ into following number system:

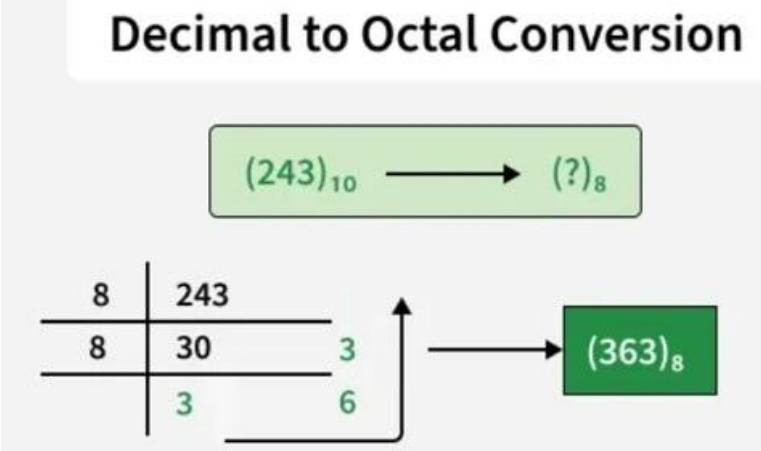
a) Binary number b) Octal number c) Hexadecimal number.

Ans: a) binary number: Decimal numbers are represented in base 10, but the Binary numbers are of base 2. Hence, to convert the decimal number to a Binary number, the base of that number is to be changed.



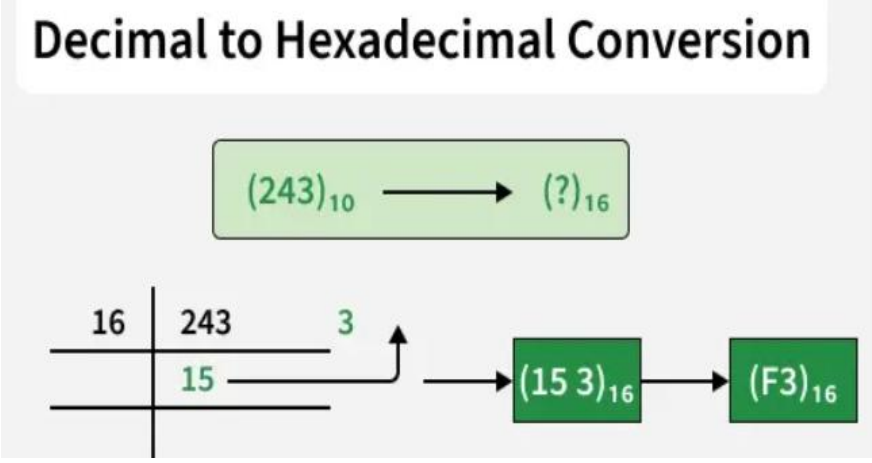
$(243)_{10} = (11110011)_2$

b) Octal number: Decimal numbers are represented in base 10, but the Octal numbers are of base 8 Hence, to convert the decimal number to a Octal number, the base of that number is to be changed.



$(243)_{10} = (363)_8$

c) Hexadecimal number: Decimal numbers are represented in base 10, but the Hexadecimal numbers are of base 16. Hence, to convert the decimal number to a Hexadecimal number, the base of that number is to be changed.



$(243)_{10} = (F3)_{16}$

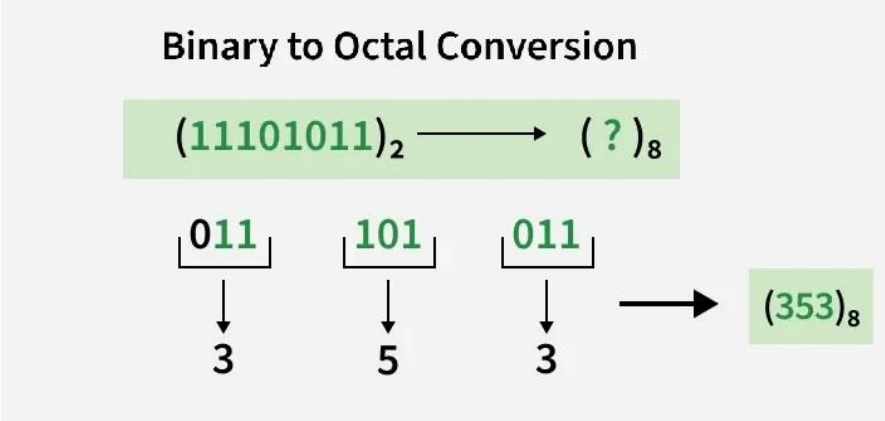
2) Convert $(11101011)_2$ into following number system:

a) Octal number b) Decimal number c) Hexadecimal number.

Ans: a) To convert $(11101011)_2$ into an octal number

Step 1: Divide the binary number into groups of three digits starting from right to left i.e. from LSB to MSB.

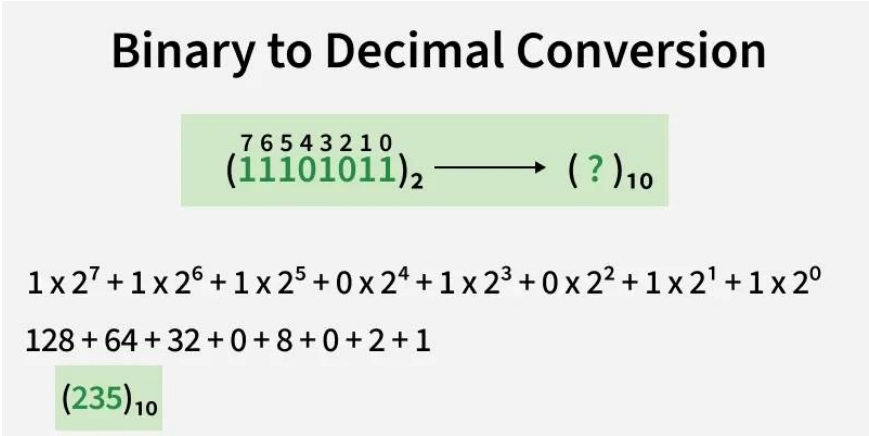
Step 2: Convert these groups into equivalent octal digits.



b) To convert $(11101011)_2$ into Decimal number:

Step 1: Multiply each digit of the Binary number with the place value of that digit, starting from right to left i.e. from LSB to MSB.

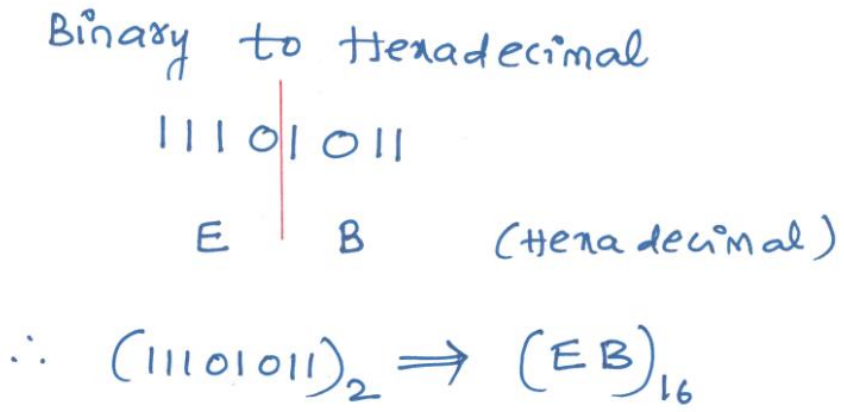
Step 2: Add the result of this multiplication and the decimal number will be formed.



c) To convert $(11101011)_2$ into Hexadecimal number

Step 1: Divide the binary number into groups of four digits starting from right to left i.e. from LSB to MSB.

Step 2: Convert these groups into equivalent Hexadecimal numbers.



3) Convert $(247)_8$ into following number system:

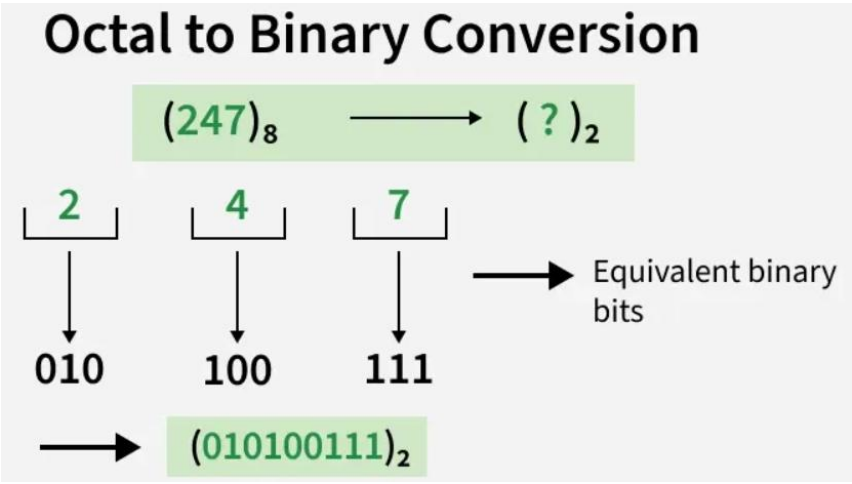
a) Binary number b) Decimal number c) Hexadecimal number.

Ans: a) Binary number:

Step 1: Write each digit of the octal number separately.

Step 2: Convert each digit into an equivalent group of three binary digits.

Step 3: Combine these groups to form the whole binary number.



b) Decimal number:

Step 1: Multiply each digit of the Octal number with the place value of that digit, starting from right to left i.e. from LSB to MSB.

Step 2: Add the result of this multiplication and the decimal number will be formed.

$$(247)_8 = 2 \times 8^2 + 4 \times 8^1 + 7 \times 8^0$$

$$= 2 \times 64 + 4 \times 8 + 7 \times 1$$

$$= 128 + 32 + 7$$

$$= 167$$

$$(247)_8 = (167)_{10}$$

c) Hexadecimal number:

Step 1: To convert Octal number into Hexadecimal, first convert that octal number into binary number.

Replace each octal digit with its 3-bit binary equivalent:

- 2 → 010
- 4 → 100
- 7 → 111

$$(247)_8=(010100111)_2$$

Step 2: After binary convertion, convert that binary number into hexadecimal number.

010100111 → 0000 1010 0111

- 0111 → 7
- 1010 → 10 → A
- 0000 → 0

$$(247)_8=(0A7)_{16}$$

4) Convert $(B2E)_{16}$ into following number system:

a) Binary number **b) Octal number** **c) Decimal number**

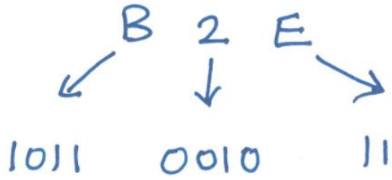
Ans: **a) Convert $(B2E)_{16}$ into Binary number:**

Step 1: Write each digit of the Hexadecimal number separately.

Step 2: Convert each digit into an equivalent group of four binary digits.

Step 3: Combine these groups to form the whole binary number.

Hexadecimal to Binary conversion



$\therefore \quad (101100101110)_2$

b) Convert $(B2E)_{16}$ into octal number:

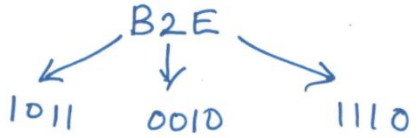
Step 1: Write each digit of the Hexadecimal number separately.

Step 2: Convert each digit into an equivalent group of four binary digits.

Step 3: Combine these groups to form the whole binary number.

Step 4: Convert each digit into an equivalent group of three binary digits.

Hexadecimal to octal conversion



101|100|101|110 (Binary Form)

5 | 4 | 5 | 6 (octal number)

$\therefore \quad (B2E)_{16} \Rightarrow (5456)_8$

c) Decimal number:

Step 1: Multiply each digit of the Hexadecimal number with the place value of that digit, starting from right to left i.e. from LSB to MSB.

Step 2: Add the result of this multiplication and the decimal number will be formed.

Hexa decimal to Decimal number

$(B2E)_{16} = B\times16^2 + 2\times16^1 + E\times16^0$

$= (11)\times16^2 + 2\times16^1 + (14)\times16^0$

$= 11\times16^2 + 2\times16 + 14\times1$

$= 2816 + 32 + 14$

$= (2862)_{10}$

5) find the 2's complement for the following

a) $(01010101)_2$ **b) $(10101101)_2$** **c) $(0110110100)_2$**

Ans:

a) $(01010101)_2$

Given that $(01010101)_2$

Step 1 : 1's complement $\Rightarrow$ 10101010

Step 2 : 2's complement $\Rightarrow$ 10101010

1 (add 1)

10101011

$\therefore$ 2's complement for given Binary number is $(10101010)_2$

b) Given that, $(10101101)_2$

Step 1 : 1's complement ⇒ 01010010

Step 2: 2's complement ⇒ 01010010
1 (add 1)
01010011

2's complement For given Binary number is

$\therefore (01010011)_2$

c) Given that, $(0110110100)_2$

Step 1 : 1's complement ⇒ 1001001011

Step 2: 2's complement ⇒ 1001001011
1111 (add 1)
1001001100

2's complement For given Binary number is

$(1001001100)_2$

6) What is Boolean Algebra? Give any five properties of Boolean Algebra.

Ans: Boolean algebra is a branch of mathematics that provides a set of operations and rules to manipulate and process binary variables.

- Boolean algebra is based on the binary number system.
- The operations in Boolean algebra are based on the three fundamental logical operations namely, AND, OR, and NOT.

i) **Identity Law :** The identity law state that in boolean algebra we have such variables that on operating with AND and OR operation we get the same result, i.e.

- A + 0 = A
- A . 1 = A

ii) **Complement law:** In this Law if a complement is added to a variable it gives 1, if a variable is multiplied with its complement it results in '0', i.e.,

- A + A' = 1
- A.A' = 0

iii) **Idempotent law:** A variable remains unchanged when it is OR or AND with itself, i.e.,

A + A = A

A.A = A

iv) **Commutative Law:** This law states that operating boolean variables A and B is similar to operating boolean variables B and A. That is,

A . B = B . A

A + B = B + A

v) **Demorgans Law:** This law state that the complement of the AND operation of two variables is equal to the OR operation of their complements, and the complement of the OR operation of two variables is equal to the AND operation of their complements. Mathematically, the laws are expressed as:

$$\overline{A.B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A}.\overline{B}$$

vi) **Associative Law:** This law state that the order of performing Boolean operator is illogical as their result is always the same. This can be understood as,

(A . B) . C = A . (B . C)

(A + B) + C = A + (B + C)

vii) Distributive Law: The expression for Distributive law is given as

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

$$A + (B \cdot C) = (A + B)(A + C)$$

viii) Inversion Law: Inversion law is the unique law of Boolean algebra this law states that, the complement of the complement of any number is the number itself.

$$(A')' = A$$

7) a) Express the Boolean function F = (x + y z) as a sum of minterms.

b) Express the Boolean function F = (x + y z) as a product of maxterms.

Ans: a) This function has three variables: *x*, *y*, and *z*. All terms must have these three variables. Thus, we need to expand the first term by AND operation it with $(y + y')(z + z')$, and we expand the second term with $(x + x')$ to get,

F = x + y z

= x (y + y') (z + z') + (x + x') y z

= x y z + x y z' + x y' z + x y' z' + x y z + x' y z

= x' y z + x y' z' + x y' z + x y z' + x y z

= m3 + m4 + m5 + m6 + m7

= Σ(3, 4, 5, 6, 7)

x	y	z	Minterms
0	0	0	m_0=x' y' z'
0	0	1	m_1=x' y' z
0	1	0	m_2=x' y z'
0	1	1	m_3=x' y z
1	0	0	m_4=x y' z'
1	0	1	m_5=x y' z
1	1	0	m_6=x y z'
1	1	1	m_7=x y z

b) First, we need to convert the function into the product-of-OR terms by using the distributive law as follows:

F = x + y z

= (x + y) (x + z) (distributive law)

= (x + y + z z') (x + y y' + z) (expand 1st term by ORing it with *z z'*, and 2nd term with *y y)*

= (x + y + z) (x + y + z') ~~(x + y + z)~~ (x + y' + z)

= M0 • M1 • M2

= Π(0, 1, 2).

x	*y*	*z*	Maxterms
0	0	0	$M_0=x+y+z$
0	0	1	$M_1=x+y+z'$
0	1	0	$M_2=x+y'+z$
0	1	1	$M_3=x+y'+z'$
1	0	0	$M_4=x'+y+z$
1	0	1	$M_5=x'+y+z'$
1	1	0	$M_6=x'+y'+z$
1	1	1	$M_7=x'+y'+z'$

8) a) Express the Boolean function F = (A+B'C+ABC) as a sum of Minterms.

b) Express the Boolean function F = A'(B'+C) as a product of Maxterms.

a) Ans:

F (A, B, C) = A + B'.C + A.B.C

= A (B + B') (C + C') + B'.C (A + A') + A.B.C

= A.B.C + A.B.C' + A.B'.C + A.B'.C' + A.B'.C + A'.B'.C + A.B.C

= A.B.C + A.B.C' + A.B'.C + A.B'.C' + A'.B'.C

F (A, B, C) = ∑ (1, 4, 5, 6, 7)

b) Ans:

$$F(A, B, C) = A'(B' + C)$$
$$= (A' + B.B' + C.C')(B' + C + A.A')$$
$$= [(A' + B)(A' + B') + C.C'][B' + C + A][B' + C + A']$$
$$= (A' + B + C.C')(A' + B' + C.C')(A + B' + C)(A' + B' + C)$$
$$= (A' + B + C)(A' + B + C')(A' + B' + C)(A' + B' + C')(A + B' + C)(A' + B' + C)$$
$$= (A' + B + C)(A' + B + C')(A' + B' + C)(A' + B' + C')(A + B' + C)$$

$$F(A, B, C) = \prod(2, 4, 5, 6, 7)$$

9) What are Logic gates? Briefly explain about Basic logic gates with symbols and truth tables.

Ans: Logic gates are the fundamental building blocks in digital electronics.

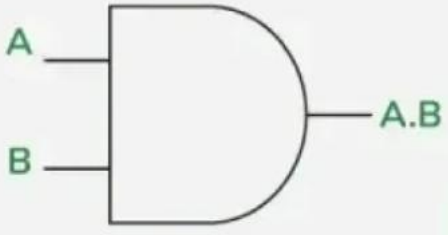
- Used to perform logical operations based on the inputs provided to it and gives a logical output that can be either high(1) or low(0).
- Basic Logic Gates :− AND Gate, OR Gate, NOT Gate

1.AND GATE:

- An AND gate is used to perform logical Multiplication of binary input.
- The Output state of the AND gate will be high (1) if both the input is high (1), else the output state will be low(0) if any of the input is low (0).
- The Boolean Expression or logic for the AND gate is the logical multiplication of inputs denoted by a full stop or single dot as :

X = A.B

- The value of X will be True when both the inputs will be True.



Truth Table

A (Input 1)	B (Input 2)	X = (A.B)
0	0	0
0	1	0
1	0	0
1	1	1

2.OR GATE:

- The OR gate is used to perform logical addition operation of binary input.
- The output state of OR gate will be high(1), if any of the input state is high(1), else output state will be low (0).
- The Boolean Expression for the OR gate is the logical addition of inputs denoted by plus sign (+) as

X= A+B



Truth Table

Input A	Input B	Output
0	0	0
0	1	1
1	0	1
1	1	1

3.NOT GATE:

- The NOT gate is one of the basic Logic Gate having only a single input and a single output. It is also known as inverter or inverting buffer.
- When the input signal is "low" the output signal is "high" and vice-versa.
- The Boolean expression of NOT Gate is as follows:

Y = Ā or **Y = A'**



Truth Table

A (Input)	Y = Ā (Output)
0	1
1	0

10) Explain briefly about Universal logic gates and Derived logic gates with symbols and truth tables.

Ans:

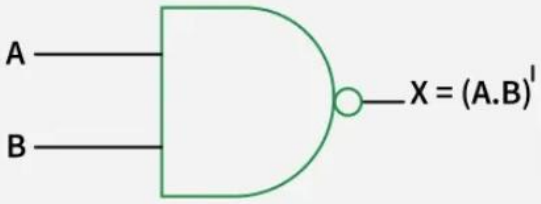
- Universal Logic Gates :– NAND Gate and NOR Gate
- Derived Logic Gates :– XOR Gate and XNOR Gate

Universal Logic Gates:

NAND GATE:

- The NAND gate is the combination of two basic logic gates AND gate and the NOT gate connected in series. It takes two or more inputs and gives only one output.
- The output of the NAND gate will give result high (1) when either of its input is high (1) or both of its input are low (0). In simple, it performs the inverted operation of AND gate.
- The Boolean Expression of NAND Gate is as follows:

$$X = \overline{A.B} \quad \text{or} \quad X = (A.B)'$$



Truth Table

Input A	Input B	X = (A.B)'
0	0	1
0	1	1
1	0	1
1	1	0

NOR GATE:

- The output state of the NOR gate will be high (1) when all the inputs are low (0).
- NOR gate returns the complement result of the OR gate. It is basically a combination of two basic logic gates i.e., OR gate and NOT gate.
- The Boolean expression of NOR gate is as follows:

$$X = \overline{A+B} \quad \text{or} \quad X = (A+B)'$$



Truth Table

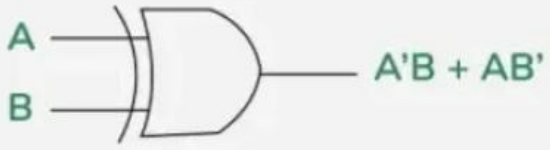
Input A	Input B	X = $\overline{(A + B)}$
0	0	1
0	1	0
1	0	0
1	1	0

Derived Logic Gates:

XOR GATE:

- It is also referred to as **Exclusive OR gate or Ex-OR gate**. it is used extensively in arithmetic logic circuits., logic comparators and error detection circuits.
- The output of the XOR gate is high (1) only when its two inputs are dissimilar i.e., if one of them is low (0) then other one will be high (1).
- The Boolean expression of XOR Gate is as follows:

$$X = A'B + AB'$$



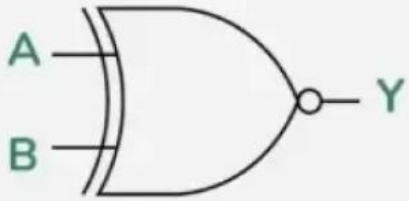
Truth Table

A (Input 1)	B (Input 2)	X = A'B + AB'
0	0	0
0	1	1
1	0	1
1	1	0

XNOR GATE:

- The XNOR is the combination of XOR gate and NOT gate. The output of the XNOR gate is high(1) when both the inputs are high (1) or low(0). In other words, the output of the XNOR gate is high(1) when both the inputs are the same.
- The following is the Boolean expression of the XNOR gate,

$Y = (A \oplus B)'$ or $\overline{(A \oplus B)}$



Truth Table

Input A	Input B	Output
0	0	1
0	1	0
1	0	0
1	1	1

MODULE-2

GATE-LEVEL MINIMIZATION TECHNIQUES

Short Question and Answers

1) What is the K-Map? what are the steps to solve expression using K-Map?

Ans: A Karnaugh map (K-map) is a visual method used to simplify Boolean expressions and truth tables.

Steps to Solve Expression using K-map:

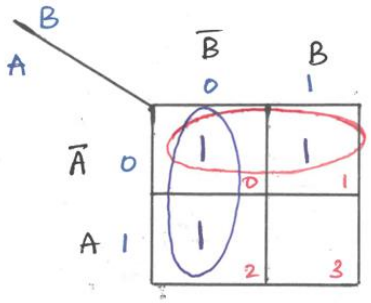
1. Select the K-map according to the number of variables.
2. Identify minterms or maxterms as given in the problem.
3. For SOP put 1's in blocks of K-map respective to the minterms (0's elsewhere).
4. For POS put 0's in blocks of K-map respective to the max terms (1's elsewhere).
5. Make rectangular groups containing total terms in power of two like 2,4,8 ..(except 1) and try to cover as many elements as you can in one group.
6. From the groups made in step 5 find the product terms and sum them up for SOP form.

2) Simplify the Boolean function F(A,B)= Σ m(0,1,2).

Ans:

Solution: Given that,

$F(A,B) = \Sigma m(0,1,2)$



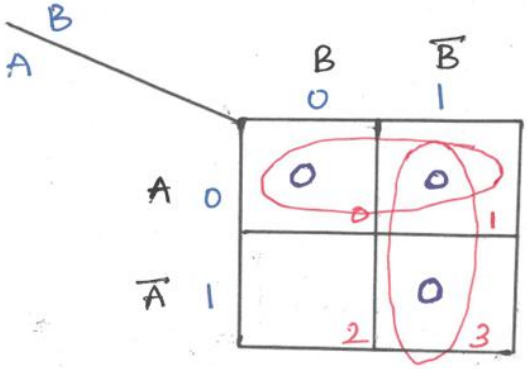
here two groups are formed, From these two groups,

$= \bar{A} + \bar{B}$

3) Simplify the Boolean function F(A,B)= ΠM(0,1,3).

Solution: Given that,

$$F(A,B) = \Pi M(0,1,3)$$



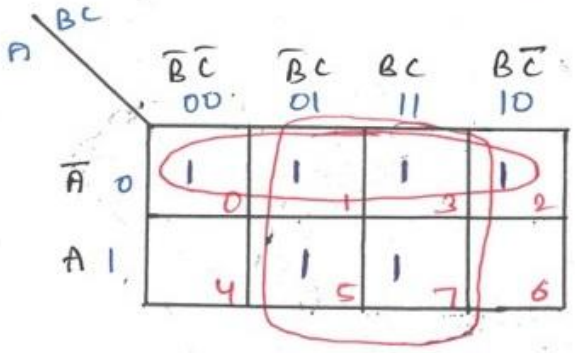
here two groups are formed, so from that two groups.

$$= (A)(\bar{B})$$

4) Simplify the Boolean function F(A,B,C)= Σ m(0,1,2,3,5,7).

Solution: Given that,

$$F(A,B,C) = \sum m(0,1,2,3,5,7)$$

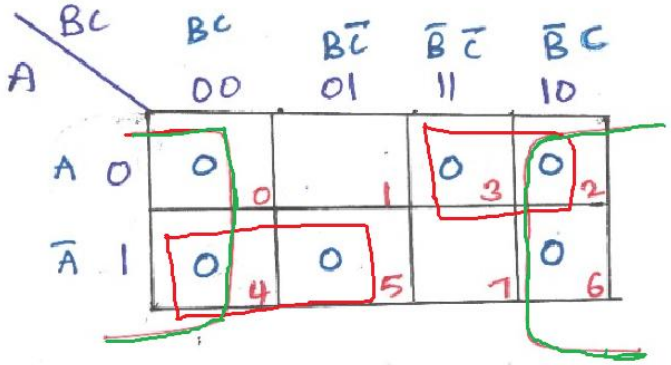


From these two groups, final expression is

$$= \bar{A} + C$$

5) Simplify the Boolean function F(A,B,C)= πM(0,2,3,4,5,6).

Solution : Given that, $F(A,B,C) = \Pi M(0,2,3,4,5,6)$

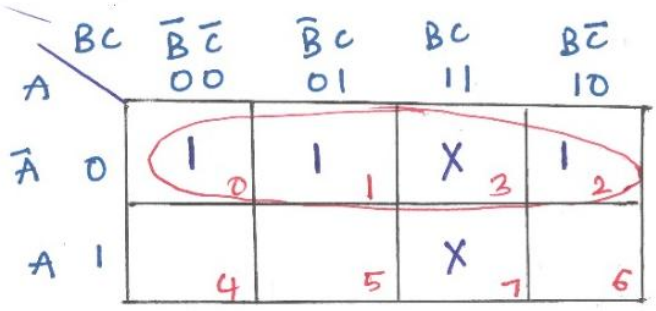


From this K-map, the simplified Expression is

$\therefore (\bar{A}+B)(A+\bar{B})(C)$

6) Simplify the following Boolean functions F(A,B,C)= Σ m(0,1,2) + d(3,7).

Given that, $F(A,B,C) = \Sigma m(0,1,2) + d(3,7)$

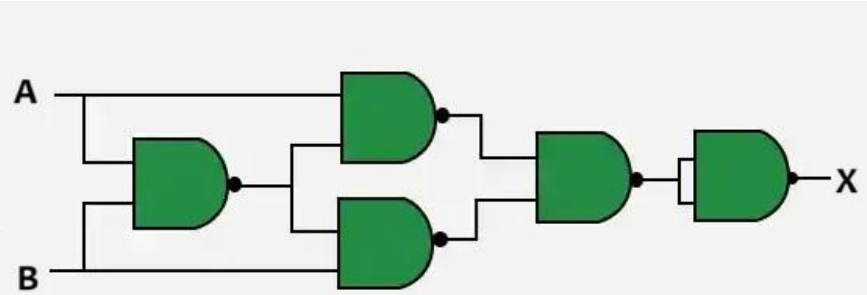


From this K-map, the Simplified Expression is

$\therefore \bar{A}$

7) Describe the implementation of XNOR gate using NAND gate.

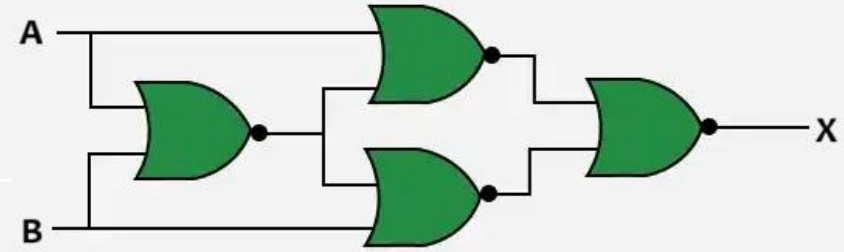
- The XNOR gate is implemented by five NAND gates.
- The first four NAND gates generate the XOR function, while the fifth NAND gate acts as a NOT gate to invert the XOR output and produce the XNOR output.
- Implementation of XNOR gate using five NAND gate as shown in the picture below:



A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

8) Describe the implementation of XNOR gate using NOR gate.

Ans: The XNOR gate can be implemented using four NOR gates. The NOR gates generate intermediate outputs that are combined to produce the XNOR output.



A	B	X
0	0	1
0	1	0
1	0	0
1	1	1

9) Simplify the Boolean Expression F= A′B′C + A′BC + AB′

Solution: F = A′B′C + A′BC + AB′

= A′C (B′+B) + AB′ [by complement law ,B + B′=1]

= A′C + AB′

10 Using Boolean algebra techniques, simplify this expression

F= A′.B.C + A.B′.C + A.B.C′ + A.B.C

Solution:

= A'.B.C + A.B'.C + A.B.C' + A.B.C

= A'.B.C + A.B'.C + A.B.C' + A.B.C + A.B.C + A.B.C

= (A'.B.C + A.B.C) + (A.B'.C + A.B.C) + (A.B.C' + A.B.C)

= (A' + A). B.C + (B' + B). C.A + (C' + C). A.B

= B.C + C.A + A.B

Long Question and Answers

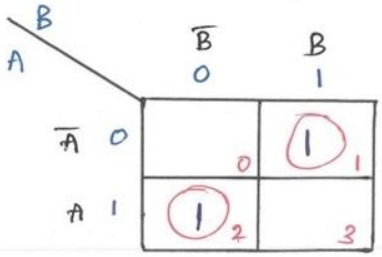
1) Simplify the Boolean functions

a) F(A,B)= Σ m(1,2).

b) F(A,B)= ΠM(0,3).

a)

Solution :- Given that, $F(A,B) = \sum m(1,2)$



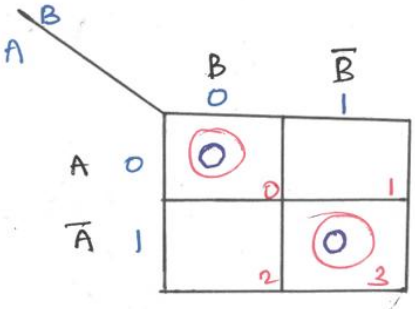
here, no groups are formed, so we are taking individual expressions

$= \bar{A}B + A\bar{B}$

b)

Solution ; Given that,

$F(A,B) = \Pi M(0,3)$



here no groups are Formed, So we are taking individual Expressions.

$= (A+B)(\bar{A}+\bar{B})$

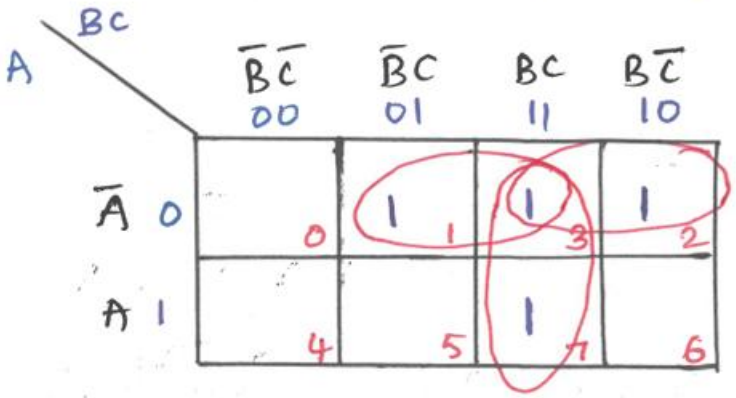
2) Simplify the Boolean functions

a) F(A,B,C)= Σ m(1,2,3,7).

b) F(A,B,C)= ΠM(0,1,3,5).

a)

Solution: Given that, $F(A,B,C) = \sum m(1,2,3,7)$

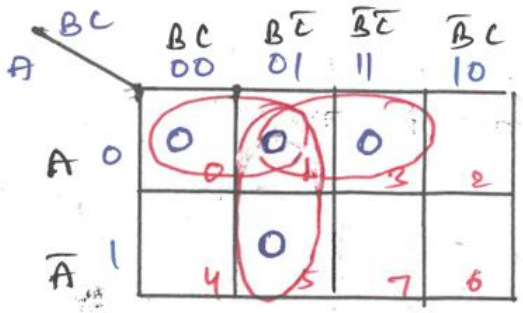


From these three groups, Final Expression is

$= \bar{A}C + \bar{A}B + BC$

b)

Solution:



$\Rightarrow (A+B)(A+\bar{C})(B+\bar{C})$

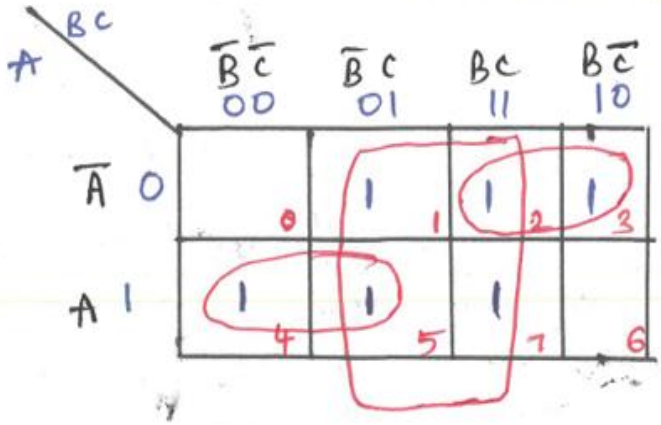
3) Simplify the Boolean functions using K-Map

i. **F(A,B,C)= Σ m(1,2,3,4,5,7).**

ii. **F(A,B,C)= πM(0,2,3,6,7).**

Ans: i)

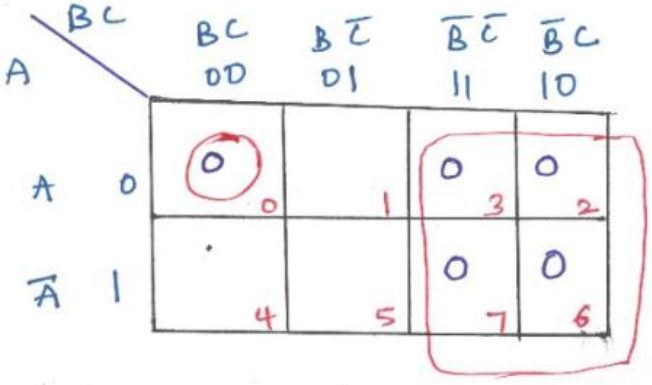
Solution: Given that, $F(A,B,C) = \sum m(1,2,3,4,5,7)$



$\Rightarrow A\bar{B} + C + \bar{A}B$

ii)

ii) Given that, $F(A,B,C) = \pi M(0,2,3,6,7)$



From this K-map, the Simplified Expression is

$\therefore (A+B+C)(\bar{B})$

4) Simplify the following Boolean Expressions

i. **F(A,B,C)= AB+(AC)'+AB'C(AB+C)**

ii. **F(A,B,C)= (A+B)(A+C)**

a) Solution: F = AB + (AC)′ + AB′C(AB + C)

= AB + A′ + C′+ AB′C.AB + AB′C.C

= AB + A′ + C′ + 0 + AB′C (B.B′ = 0 and C.C = C)

= ABC + ABC′ + A′ + C′ + AB′C [AB = AB(C + C′) = ABC + ABC′]

= AC(B + B′) + C′(AB + 1) + A′

= AC + C′+A′ (B + B′ = 1 and AB + 1 = 1)

= AC + (AC)′

= 1

b) Solution: F(A,B,C) = (A+B) (A+C)

= A. A+A.C+B. A+B.C

= A+A.C+B. A+B.C [Applying Idempotent Law]

=A(1+C) +B. A+B.C [Applying dominance Law]

=A+B.A+B.C

=A(1+B)+B.C

=A+B.C

5) Simplify the following Boolean Expressions

i. **F(A,B,C,D,E)= A+A'B+A'B'C+A'B'C'D+A'B'C'D'E**

ii. **F(A,B,C)= F = (A + B).(A + B').(A' + B')**

i.Ans:

Given that

$F(A,B,C,D,E) = A + A'B + A'B'C + A'B'C'D + A'B'C'D'E$

$= (A+B) + A'B'C + A'B'C'D + A'B'C'D'E$

$= (A+B) + (A+B)'C + A'B'C'D + A'B'C'D'E$

$= ((A+B) + (A+B)')(A+B+C) + A'B'C'D + A'B'C'D'E$

$= (A+B+C) + A'B'C'D + A'B'C'D'E$

$= (A+B+C) + (A+B+C)'D + A'B'C'D'E$

$= (A+B+C+D) + A'B'C'D'E$

$= (A+B+C+D) + (A+B+C+D)'E$

$= A+B+C+D+E$

Simplified Expression for Given Function is

$A+B+C+D+E$

ii) Given that, $F(A,B,C) = (A+B)(A+B')(A'+B')$

$= (A+B)(A+B')(A'+B')$

$= (A\cdot A + A\cdot B' + AB + BB')(A'+B')$

$= (A + AB' + AB)(A'+B')$

$= (A(1+B') + AB)(A'+B')$

$= (A + AB)(A'+B')$

$= (A(1+B))(A'+B')$

$= A(A'+B')$

$= A\cdot A' + A\cdot B'$

$= A\cdot B'$

Simplified Expression for Given function is

$A\cdot B'$.

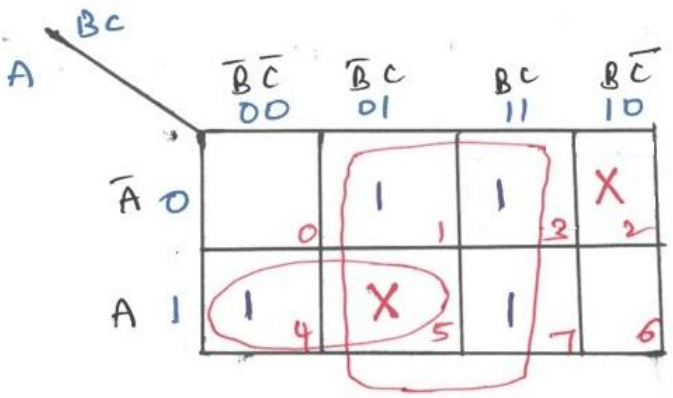
6) Simplify the following Boolean functions

i. **F(A,B,C)= Σ m(1,3,4,7) + d(2,5).**

ii. **F(A,B,C)= ΠM(0,1,3,6)+ d(5,7).**

i.

Solution:

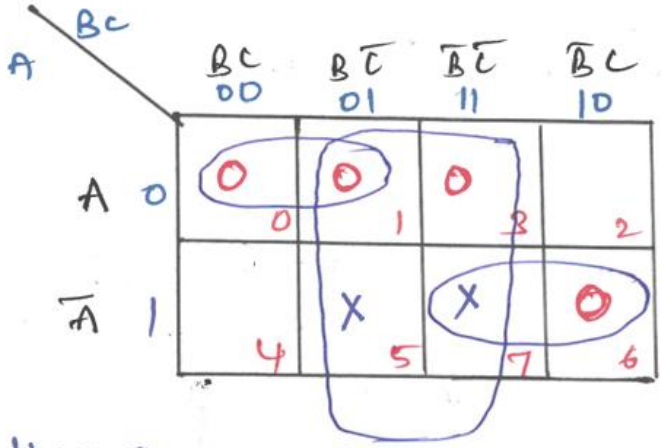


From these three Groups, final Expression is

$= A\bar{B} + C$

a)

Solution:



From these groups, final Expression is

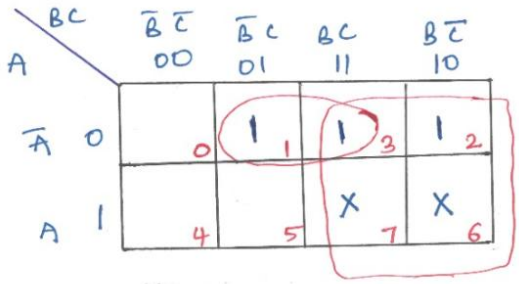
$= (A+B)(\bar{C})(\bar{A}+\bar{B})$

7) Simplify the following Boolean functions

i. **F(A,B,C)= Σ m(1,2,3,4,5) + d (6,7).**

ii. **F(A,B,C)= πM(0,3,5)+ d (1,4,6).**

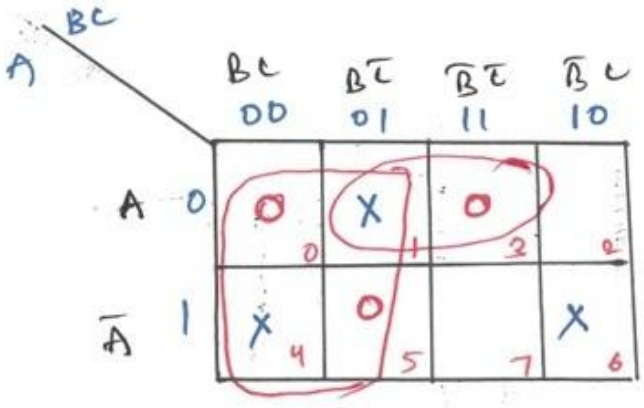
i) Given that, $F(A,B,C) = \Sigma m(1,2,3,4,5) + d(6,7)$



Here two groups are formed, the simplified expression is $\bar{A}C + B$

ii.

Solution:



⇒ from these two groups, final expression is

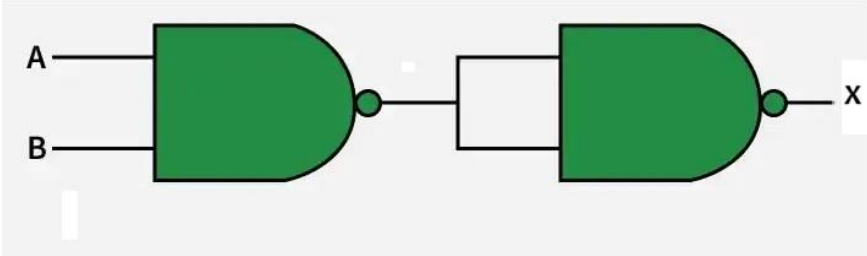
$= (B)(A + \bar{C})$

8) Describe the implementation of the following gates using NAND gate

i. AND gate b) OR gate c) NOT gate

Ans: a) Implementation of AND Gate using NAND Gates:

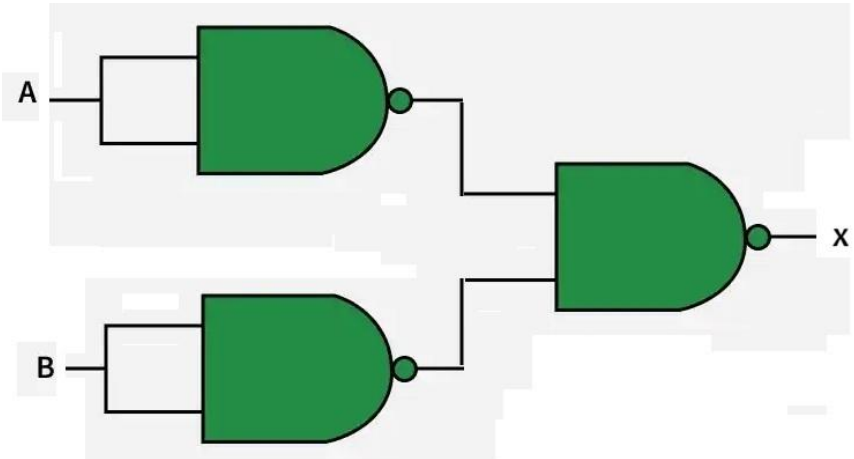
The AND gate is implemented by using two NAND gates where the output is HIGH only when both inputs are HIGH. The first NAND gate performs a standard NAND functions, the second NAND gate has both inputs tied to the output of the initial NAND gate as shown below.



Input		Output
A	B	X=A.B
0	0	0
0	1	0
1	0	0
1	1	1

b) Implementation of OR Gate using NAND Gates:

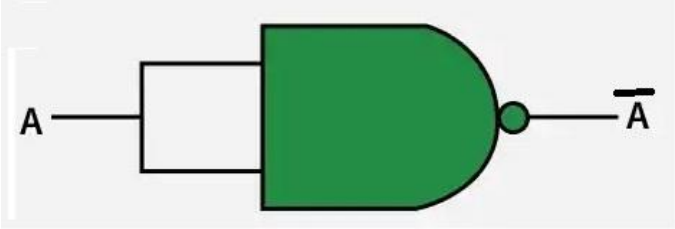
- It is implemented by using three NAND gates. The inputs are first inverted using two NAND gates having their inputs probed in parallel and then the inverted outputs are connected with the input of third NAND gate. The last NAND gate effectively performs the inverted signals to give the required OR gate function.
- The OR gate can be implemented using the NAND gate as below:



Input		Output
A	B	X =A+B
0	0	0
0	1	1
1	0	1
1	1	1

c) Implementation of NOT Gate using NAND Gates:

- A single NAND gate with 'a' and 'b' inputs connected respond to a NOT gate.
- Implementation of NOT gate using a single NAND gate as shown in the picture below:



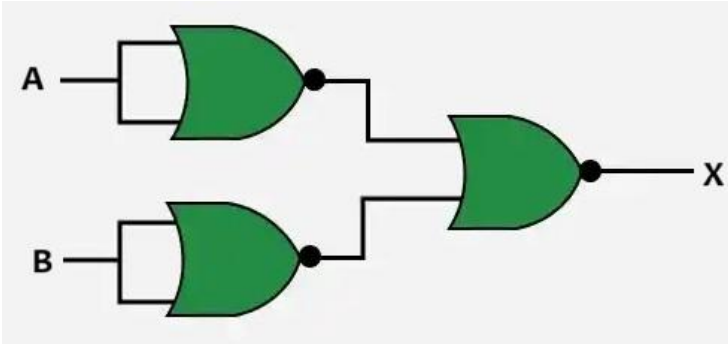
A (Input)	$\overline{A}$ (Output)
0	1
1	0

9) Describe the implementation of the following gates using NOR gate

i. AND gate b) OR gate c) NOT gate

a) Implementation of AND gate using NOR Gates:

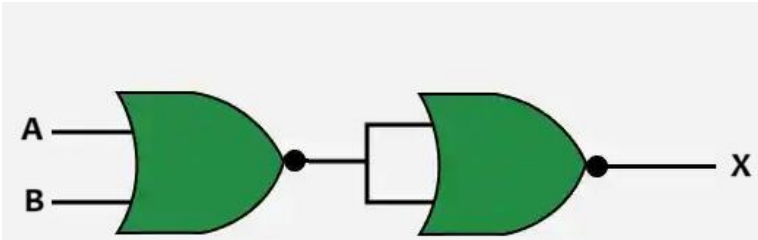
- AND gate is implemented by using three NOR gates.
- Implementation of AND gate using only NOR gates as shown below



A	B	X
0	0	0
0	1	0
1	0	0
1	1	1

b) Implementation of OR gate using NOR Gates:

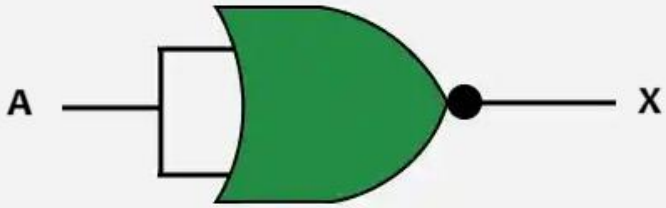
- OR gate is implemented by using two NOR gates.
- Implementation of OR gate using only NOR gates as shown below**:**



A	B	X
0	0	0
0	1	1
1	0	1
1	1	1

c) Implementation of NOT gate Using NOR Gates:

- NOT gate is implemented by using one NOR gate.
- Implementation of NOT gate using a single NOR gate as shown in the picture below:

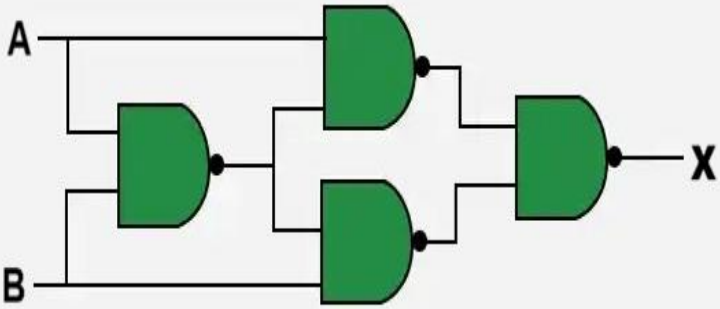


A	X
0	1
1	0

10) Describe the implementation XOR Gate using a) NAND Gate b) NOR Gate

Ans: **<u>Implementation of XOR Gate using NAND gates:</u>**

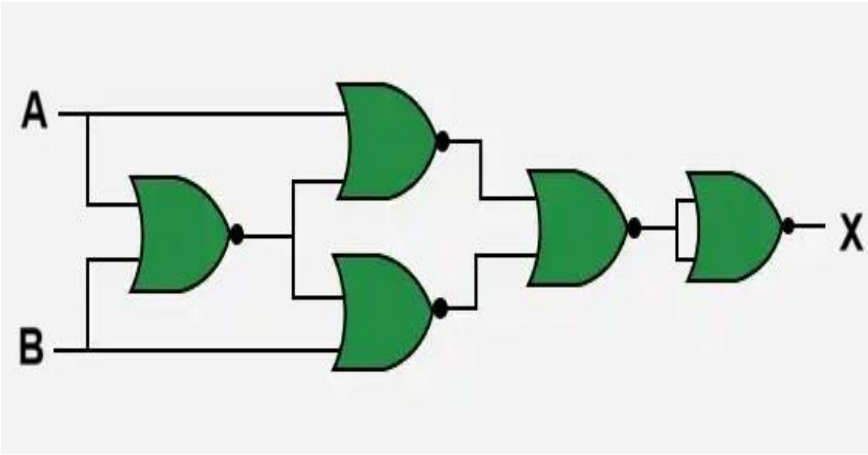
- XOR gate can be implemented using four NAND gates. The first NAND gate generates an intermediate output, while the remaining NAND gates combine the input signals to produce the XOR output.
- Implementation of XOR gate using four NAND gate as shown in the picture below:



A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Implementation of XOR gate Using NOR Gates:

- The XOR gate is implemented by five gates and these gates are the NOR gates.
- Implementation of XOR gate using five NOR gate as shown in the picture below:



A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

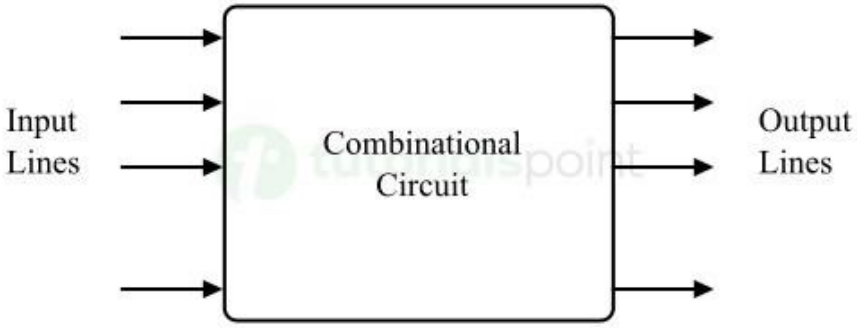
MODULE-3

COMBINATIONAL CIRCUITS

Short Questions and Answer

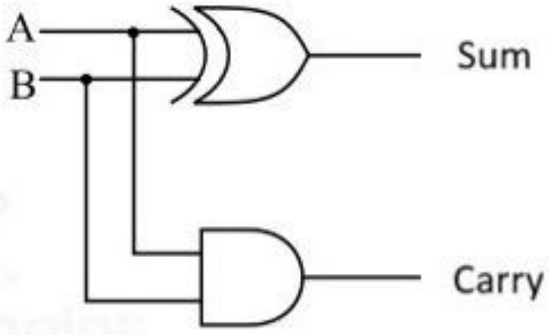
1) What is a Combinational logic circuit? Draw the Block diagram of combinational circuit.

Ans: A combinational circuit, also called a combinational logic circuit, is a digital electronic circuit whose output is determined by present inputs only.



2) Draw the Half adder circuit diagram.

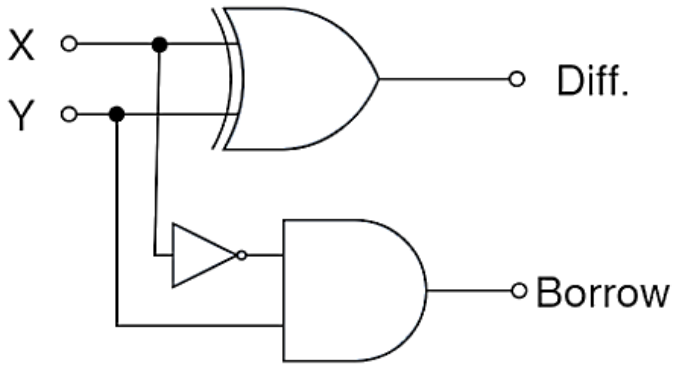
Ans:



- The expression for sum S in half adder is: **S = A ⊕ B**
- The expression for carry C in half adder is: **C = AB**

3) Draw the Half Binary subtractor circuit diagram.

Ans:



- The expression for difference D in half subtractor is: **D = X ⊕ Y**
- The expression for borrow B in half subtractor is: **B = $\bar{X}$ Y**

4) Write the differences between Binary adder and BCD adder.

Ans:

Feature	Binary Adder	BCD Adder
1. Operation	Adds standard base-2 binary numbers.	Adds 4-bit BCD digits (decimal 0 to 9).
2. Correction Logic	No correction required; raw sum is final.	Requires a correction factor of 6 (0110) if the sum exceeds 9.
3. Circuit Complexity	Simpler structure using cascaded Full Adders.	More complex structure requiring binary adders and detection logic gates.
4.Applications	Used in general-purpose computer processing and standard digital computations.	Used in specialized digital systems, calculators

5) Perform BCD addition of $(1000)_2$ and $(0111)_2$ and write the corrected BCD result.

Ans:

	7	0	1	1	1	← BCD for 7
+	8	1	0	0	0	← BCD for 8
	15	1	1	1	1	← Invalid BCD number

The sum 1 1 1 1 is an invalid BCD number. This has occurred because the sum of the two digits exceeds 9. Whenever this occurs the sum has to be corrected by the addition of six (0110) in the invalid BCD number, as shown below

	7	0	1	1	1	← BCD for 7
+	8	1	0	0	0	← BCD for 8
	15	1	1	1	1	← Invalid BCD number
+		0	1	1	0	← Add 6 for correction
		0	0	0	1	0 1 0 1 ← BCD for 15
			1			5

6) Define a magnitude comparator. What are its three possible outputs?

Ans: A magnitude comparator is a combinational circuit that compares two numbers, A and B, and determines their relative magnitudes.

- For two binary inputs **A** and **B**, the three possible outputs are:
  1. **A > B** — A is greater than B
  2. **A = B** — A is equal to B
  3. **A < B** — A is less than B

7) What is 1-bit comparator and draw its truth table.

Ans: A 1-bit comparator is a combinational logic circuit that compares two 1-bit binary numbers, A and B, and determines whether:

- A > B
- A = B
- A < B

Truth table of 1- Bit comparator:

INPUTS		OUTPUTS		
A	B	Y_1 (A<B)	Y_2 (A=B)	Y_3 (A>B)
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

8) Write the difference between Encoder and Decoder.

Ans:

S.No	Encoder	Decoder
1	Encoder converts multiple input lines into fewer output lines.	Decoder Converts fewer input lines into multiple output lines.
2	It performs encoding operation.	It performs decoding operation.
3	For an 2^n-to-n encoder, there are 2^n inputs and nn outputs.	For an n-to-2^n decoder, there are n inputs and 2^n outputs.
4	Usually only one input is active at a time.	Usually one output is active for a given input combination.
5	Example: 8-to-3 Encoder.	Example: 3-to-8 Decoder.

9) Define a multiplexer (MUX). Why is it called a data selector?

Ans: A multiplexer, also referred to as MUX, is a combinational logic circuit that is designed to accept multiple input signals and transfer only one of them through the output line.

- In simple words, a multiplexer is a digital logic device that selects one-out-of-N (N = 2^n) input data sources and transmits the selected data to a single output line.
- The multiplexer is also called **data selector** as it selects one from several.

10) Write the difference between multiplexer and demultiplexer.

Ans:

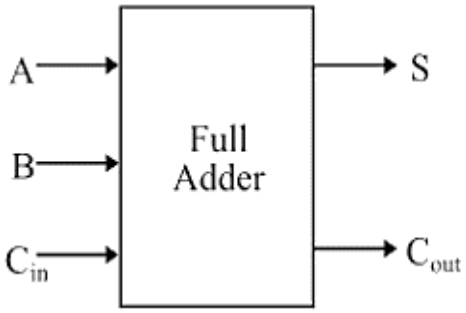
Multiplexer (MUX)	**Demultiplexer (DEMUX)**
Multiplexer selects one input from many inputs and sends it to a single output.	Demultiplexer selects one output from many outputs and sends the input to the selected output.
It is a many-to-one circuit.	It is a one-to-many circuit.
MUX uses select lines to select one input.	DEMUX uses select lines to select one output.
MUX used for data selection.	DEMUX used for data distribution.
Example: 4:1 MUX has 4 inputs and 1 output.	Example: 1:4 DEMUX has 1 input and 4 outputs.

Long Question and Answers:

1) Explain Full adder with logical diagram and truth table.

Ans:

- Full Adder is a combinational circuit that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C_{in}.
- The output carry is designated as C_{out} and the normal output is designated as S which is SUM.
- **BLOCK DIAGRAM:**



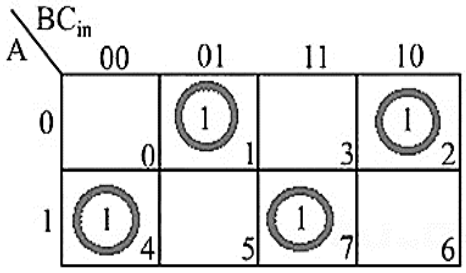
TRUTH TABLE:

Inputs			**Outputs**	
A	**B**	**C_{in}**	**Sum(S)**	**Carry (C_{out})**
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

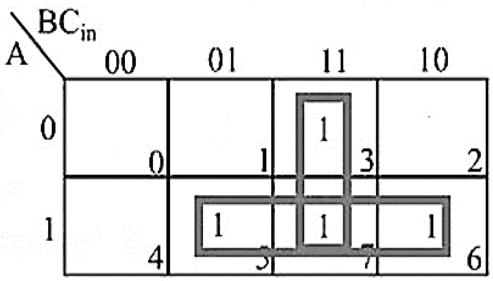
K-Map for Full Adder:

K-Map (Karnaugh Map) is a method for simplifying binary complex Boolean algebraic expressions.

For Sum (S)



For Output Carry (C_{out})



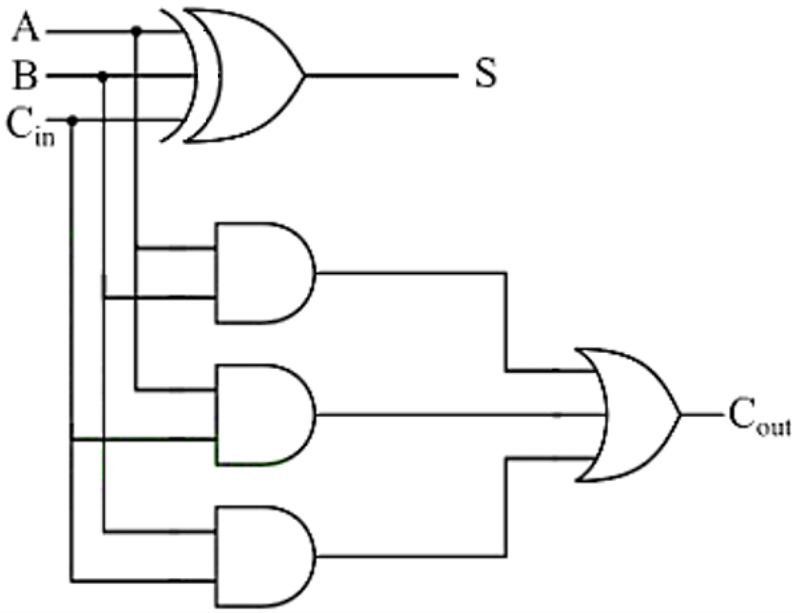
$S = A'B'C_{in} + A'BC'_{in} + AB'C'_{in} + ABC_{in}$

$= A \oplus B \oplus C_{in}$

$C_{out} = AB + AC_{in} + BC_{in}$

- The equation obtained by the EX-OR gate is the sum of the binary digits. While the output obtained by AND gate is the carry obtained by addition.
  - Sum = $(A \oplus B) \oplus C$
  - Carry-out = A.B + A.C+ B.C

LOGIC DIAGRAM:

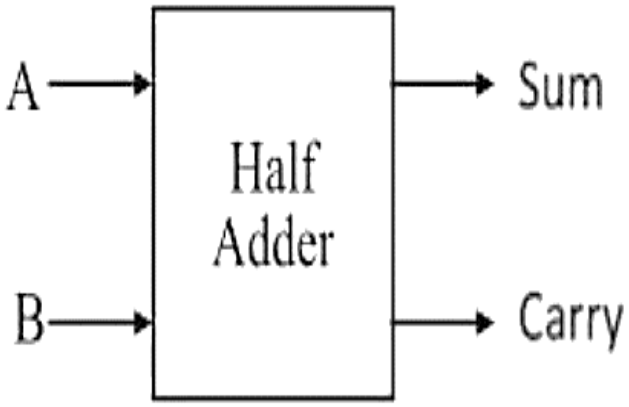


2) Explain Half Binary adder with logic diagram and truth table.

Ans:

- Half Adder is a combinational logic circuit that has two inputs: A and B, which add two input digits and generate a carry and a sum.
- There will be no forwarding of carry addition because there is no logic gate to process that. Thus, this is called the Half Adder circuit.
-

BLOCK DIAGRAM OF HALF ADDER:

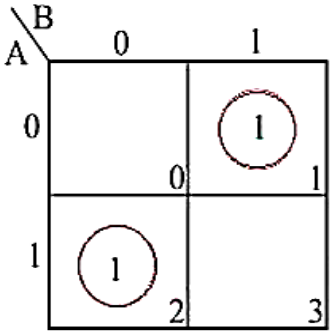


TRUTH TABLE OF HALF ADDER:

Truth Table			
Input		Output	
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

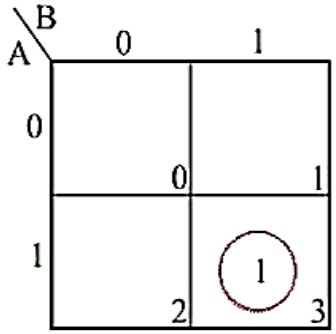
K-MAP SIMPLIFICATION:

For Sum (S)



$S = A'B + AB'$

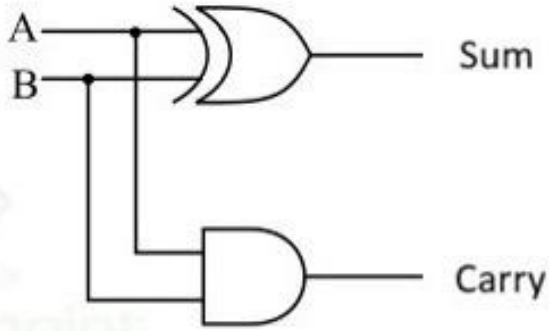
For Carry (C)



$C = A\,B$

- The expression for sum S in half adder is: **S = A ⊕ B**
- The expression for carry C in half adder is: **C = AB**

LOGIC DIAGRAM:



3) Explain Full Binary Subtractor with logic diagram and truth table.

Ans:

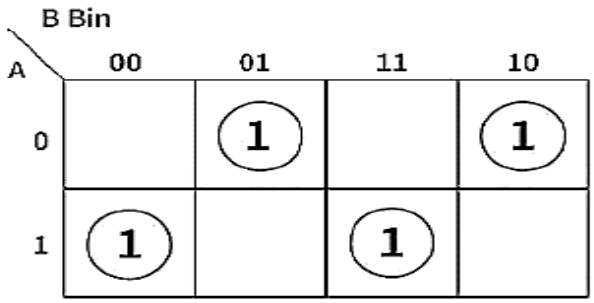
- A Full Binary Subtractor has an additional input of "Borrow-in", as compared to Half Binary Subtractor, for taking into account "Borrow-out" from the previous Half Subtractor.
- It contains three inputs(A, B, Bin) and produces two outputs (D, Bout).

Truth Table of Full Subtractor:

Inputs			Outputs	
A	B	B_{in}	D	B_{out}
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

K-MAP OF FULL SUBTRACTOR:

For Difference:

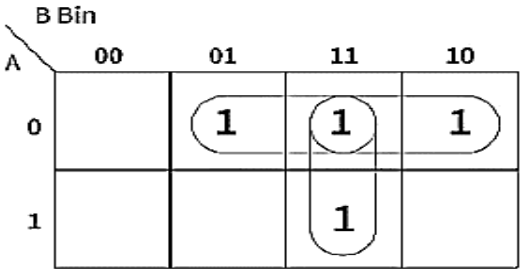


D=A'B'Bin + AB'Bin' + A'Bin' + ABBin

The final simplified expression for the difference in a full subtractor is:

$$D = (A \oplus B) \oplus Bin$$

For Borrow,

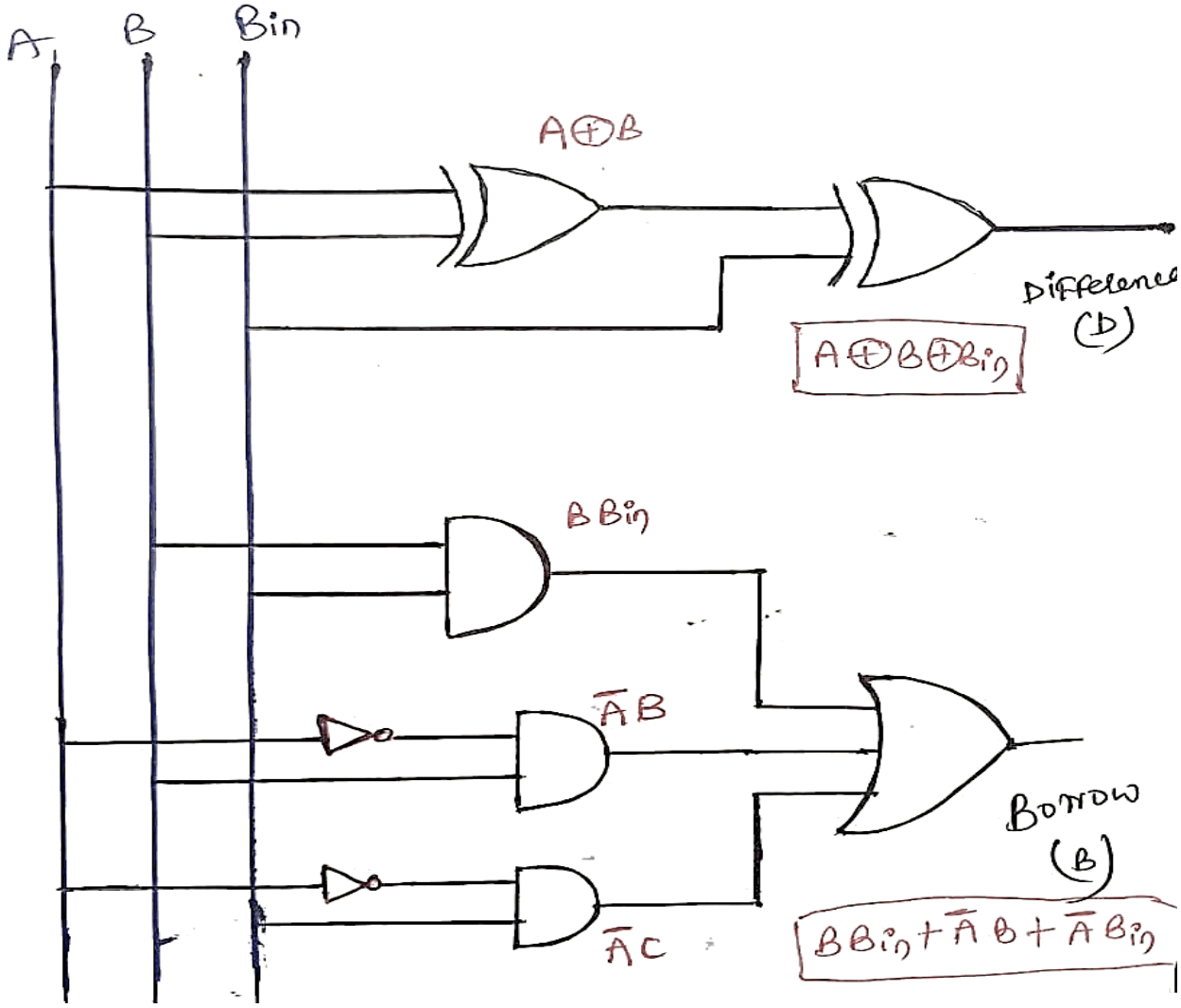


Bout=A'Bin + A'B + BBin

The final simplified expression for borrow in a full subtractor is:

Borrow (B) = BBin + $\bar{A}$B + $\bar{A}$Bin

Logic Diagram of Full Subtractor:



4) Explain Half Binary Subtractor with logic diagram and truth table.

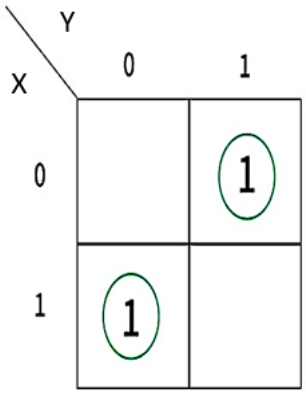
Ans:

- A Half Binary Subtractor is a combinational logic circuit that performs subtraction on two single binary digits (bits) producing Difference (D) and Borrow (B) bits.
- A Half Binary Subtractor can be constructed by combining an Exclusive-OR (XOR) gate and borrow bit generation.
- The resulting Truth Table of this Half Binary Subtractor along with the logic circuit is shown below:

Truth Table			
X	Y	DIFF.	BORR.
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

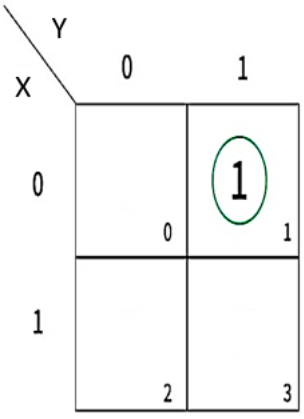
K- Map Simplification:

For difference:



Difference= X ⊕ Y

For Borrow:

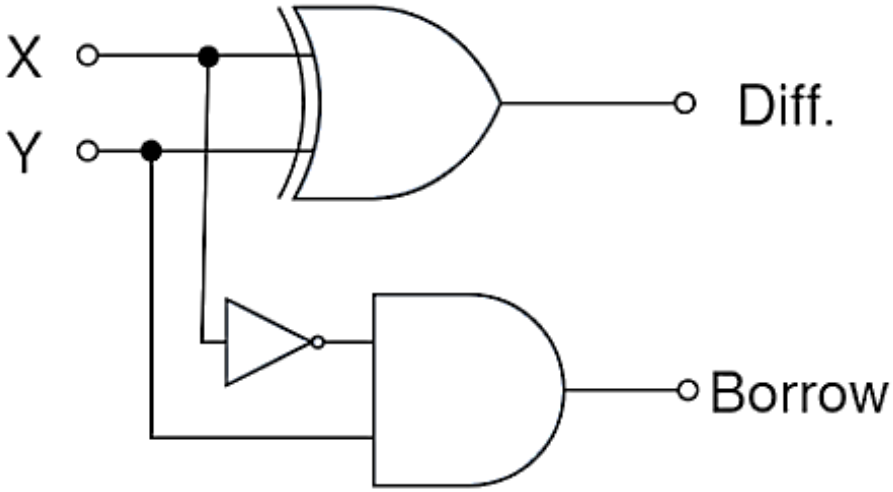


Borrow= $\bar{X}$ Y

Logic Diagram:

- The expression for difference D in half subtractor is: **D = X ⊕ Y**

- The expression for borrow B in half subtractor is: **B = $\bar{X}$ Y**



5) Explain the design and operation of a BCD Adder with a suitable block diagram.

Ans:

- A BCD (Binary Coded Decimal) adder is a digital circuit that adds two BCD digits and produces a sum digit also in BCD.
- BCD numbers use 10 digits, i.e, 0 to 9 which are represented in the binary form 0 0 0 0 to 1 0 0 1, i.e. each BCD digit is represented as a 4-bit binary number.

Example: **we write BCD number 526, it can be represented as**

5	2	6
↓	↓	↓
0 1 0 1	0 0 1 0	0 1 1 0

The BCD addition procedure as follows :

1. Add two BCD numbers using ordinary binary addition.
2. If four-bit sum is equal to or less than 9, no correction is needed. The sum is in proper BCD form.
3. If the four-bit sum is greater than 9 or if a carry is generated from the four-bit sum, the sum is invalid.
4. To correct the invalid sum, add $(0110)_2$ to the four-bit sum. If a carry results from this addition, add it to the next higher-order BCD digit.

Example 1: Let us consider additions of 3 and 6 in BCD.

	6	0	1	1	0	← BCD for 6
+	3	0	0	1	1	← BCD for 3
	9	1	0	0	1	← BCD for 9

The addition is carried out as in normal binary addition and the sum is 1001, which is BCD code for 9.

Example 2: Let us consider addition of 8 and 9 in BCD:

	8				1	0	0	0	← BCD for 8
+	9				1	0	0	1	← BCD for 9
	17	0	0	0	1	0	0	0	1	← Incorrect BCD result

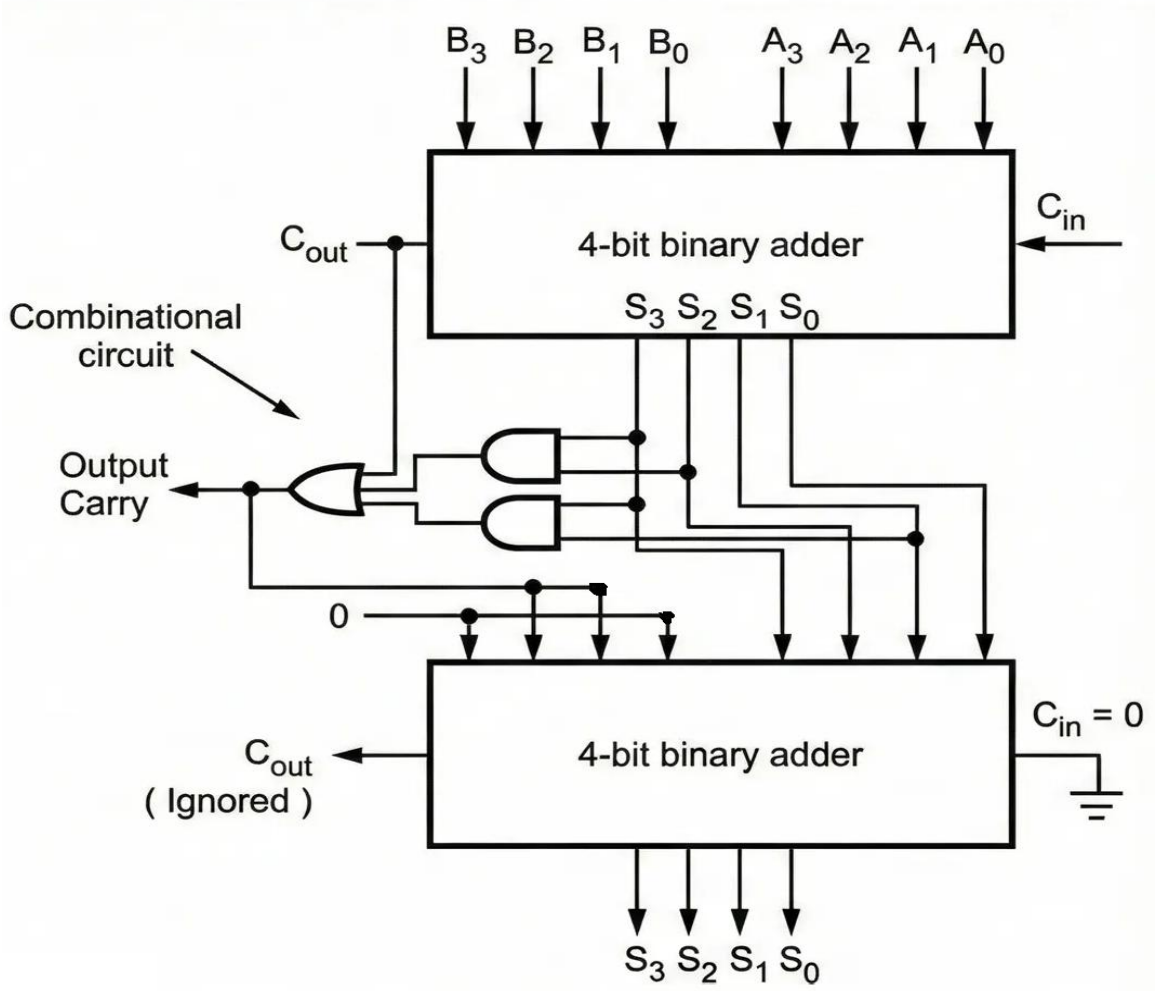
In this, case, result (0001 0001) is valid BCD number, but it is incorrect. To get the correct BCD result correction factor of 6 has to be added to the least significant digit sum, as shown below

	8				1	0	0	0	← BCD for 8
+	9				1	0	0	1	← BCD for 9
	17	0	0	0	1	0	0	0	1	← Incorrect BCD result
	+	0	0	0	0	0	1	1	0	← Add 6 for correction
		0	0	0	1	0	1	1	1	← BCD for 17

Thus to implement BCD Adder Circuit we require :

- 4-bit binary adder for initial addition
- Logic circuit to detect sum greater than 9 and
- One more 4-bit adder to add 0110_2 in the sum if sum is greater than 9 or carry is 1.

BLOCK DIAGRAM:



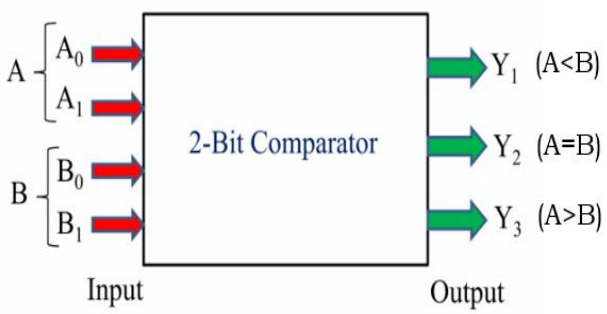
BCD Adder Block Diagram

6) Design and explain a 2-bit Comparator with its truth table and logic gates.

Ans:

- A digital combinational circuit used to compare the magnitudes of two 2-bit binary numbers.
- It consists of four inputs and three outputs to generate less than, equal to, and greater than.

The block diagram of a typical 2-bit magnitude comparator is shown in the following figure:

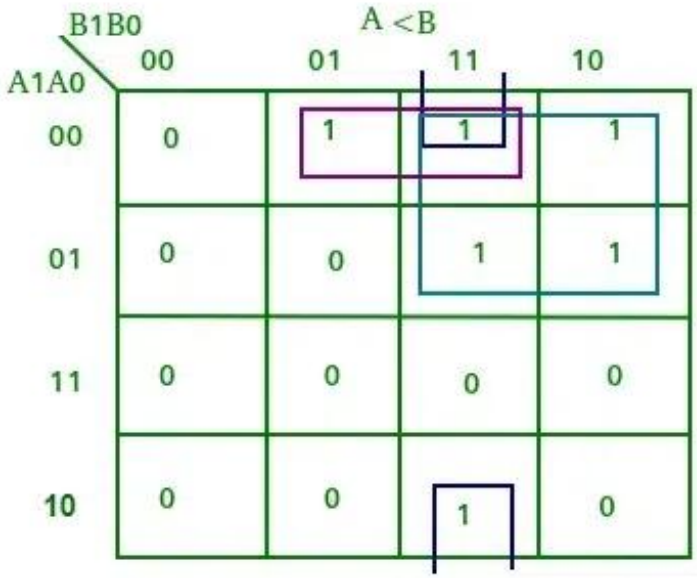


- Here, the lines A_0A_1 and B_0B_1 represents two 2-bit binary number inputs and three output lines represents the less than, equal to, and greater than.
- The truth table for a 2-bit comparator is given below:

Inputs				**Outputs**		
A_1	**A_0**	**B_1**	**B_0**	**A < B**	**A = B**	**A > B**
0	0	0	0	0	1	0
0	0	0	1	1	0	0
0	0	1	0	1	0	0
0	0	1	1	1	0	0
0	1	0	0	0	0	1
0	1	0	1	0	1	0

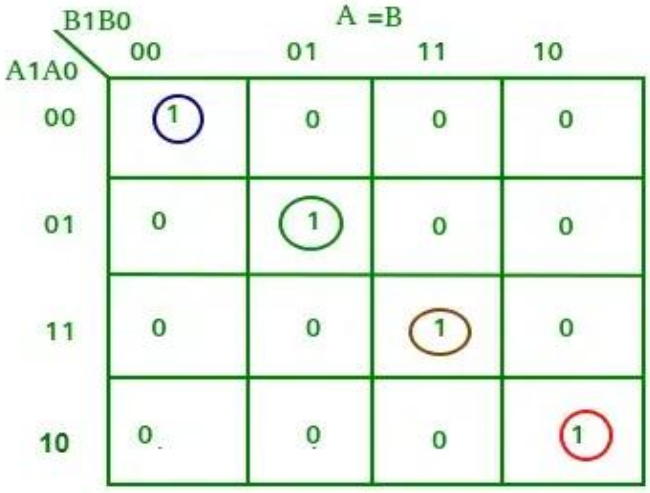
0	1	1	0	1	0	0
0	1	1	1	1	0	0
1	0	0	0	0	0	1
1	0	0	1	0	0	1
1	0	1	0	0	1	0
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	0	1	0

K- map of Output A<B:



$$Y_1 = \overline{A_1}\,\overline{A_0}\,B_0 + \overline{A_1}\,B_1 + \overline{A_0}\,B_1\,B_0$$

K- map of Output A=B:



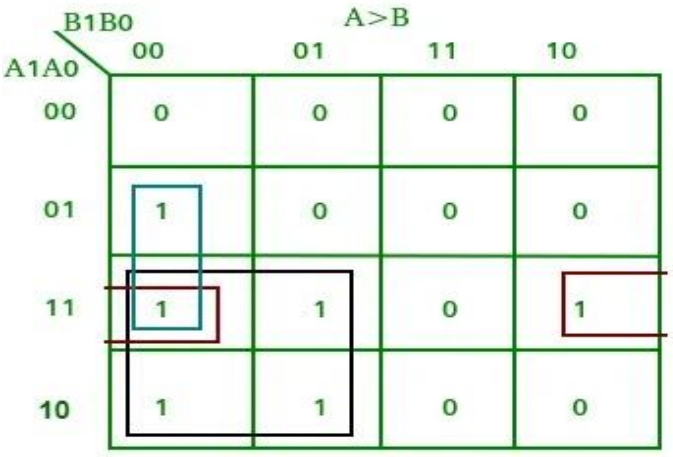
$$Y_2 = \overline{A_1}\,\overline{A_0}\,\overline{B_1}\,\overline{B_0} + \overline{A_1}\,A_0\,\overline{B_1}\,B_0 + A_1A_0B_1B_0 + A_1\,\overline{A_0}\,B_1\,\overline{B_0}$$

$$Y_2 = \overline{A_0}\,\overline{B_0}(\overline{A_1}\,\overline{B_1} + A_1B_1) + A_0B_0(\overline{A_1}\,\overline{B_1} + A_1B_1)$$

$$Y_2 = (\overline{A_1}\,\overline{B_1} + A_1B_1)\ (\overline{A_0}\,\overline{B_0} + A_0B_0)$$

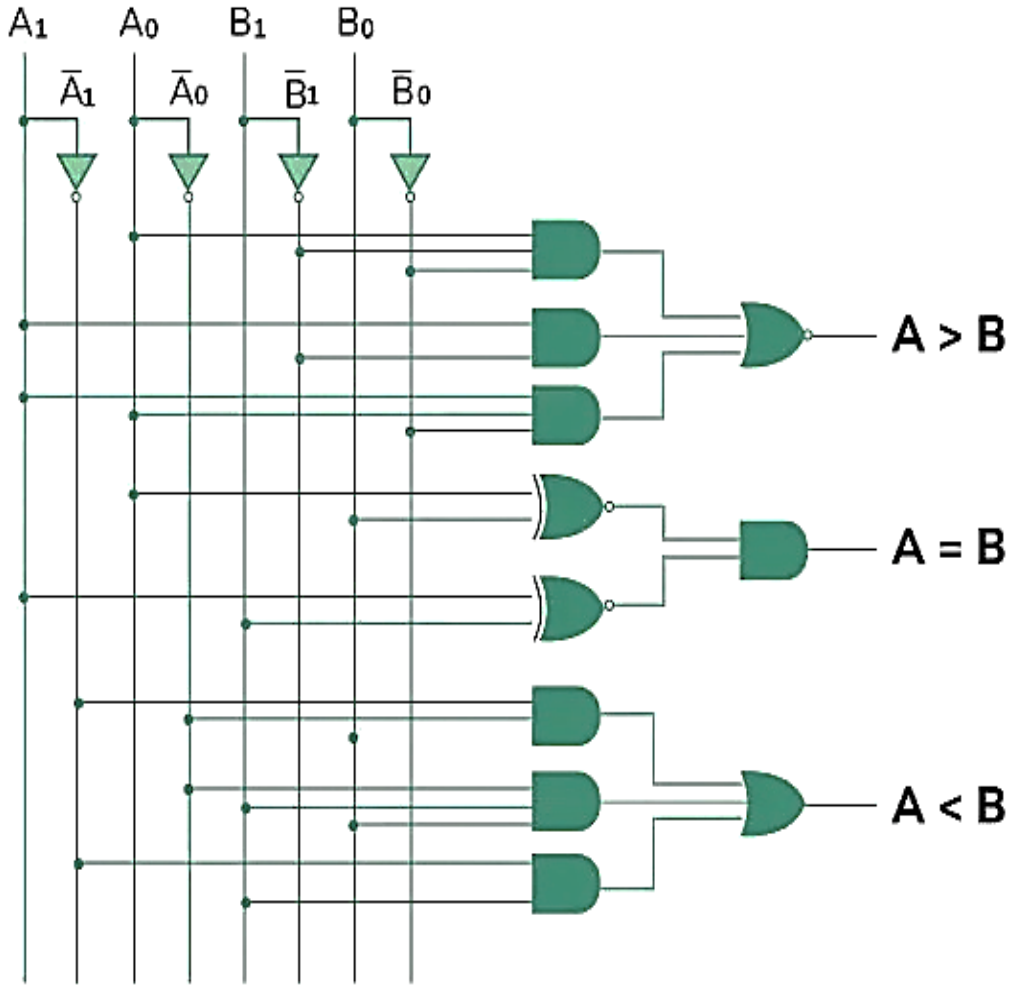
$$Y_2 = (A_1 \oplus B_1)\,(A_0 \oplus B_0)$$

K- map of Output A>B:



$$Y_3 = A_0\,\overline{B_1}\,\overline{B_0} + A_1\,\overline{B_1} + A_1A_0\,\overline{B_0}$$

LOGIC DIAGRAM:

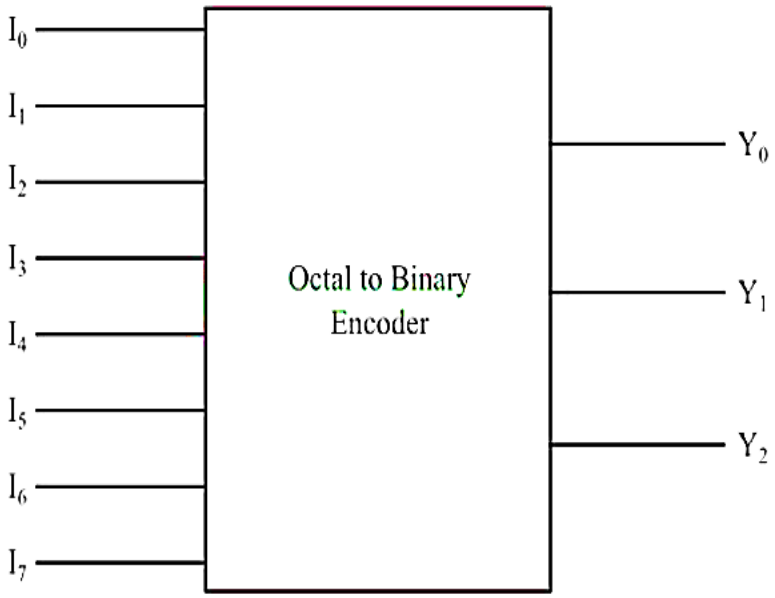


7) Design and explain 8x3 Encoder with its truth table and logic diagram.

Ans:

The 8 to 3 Encoder or octal to Binary encoder consists of 8 inputs: I_7 to I_0 and 3 outputs: Y_2, Y_1 , Y_0.

Each input line corresponds to each octal digit value and three outputs generate corresponding binary code. The figure below shows the logic symbol of octal to the binary encoder.



The following truth table describes the working of an octal to binary encoder:

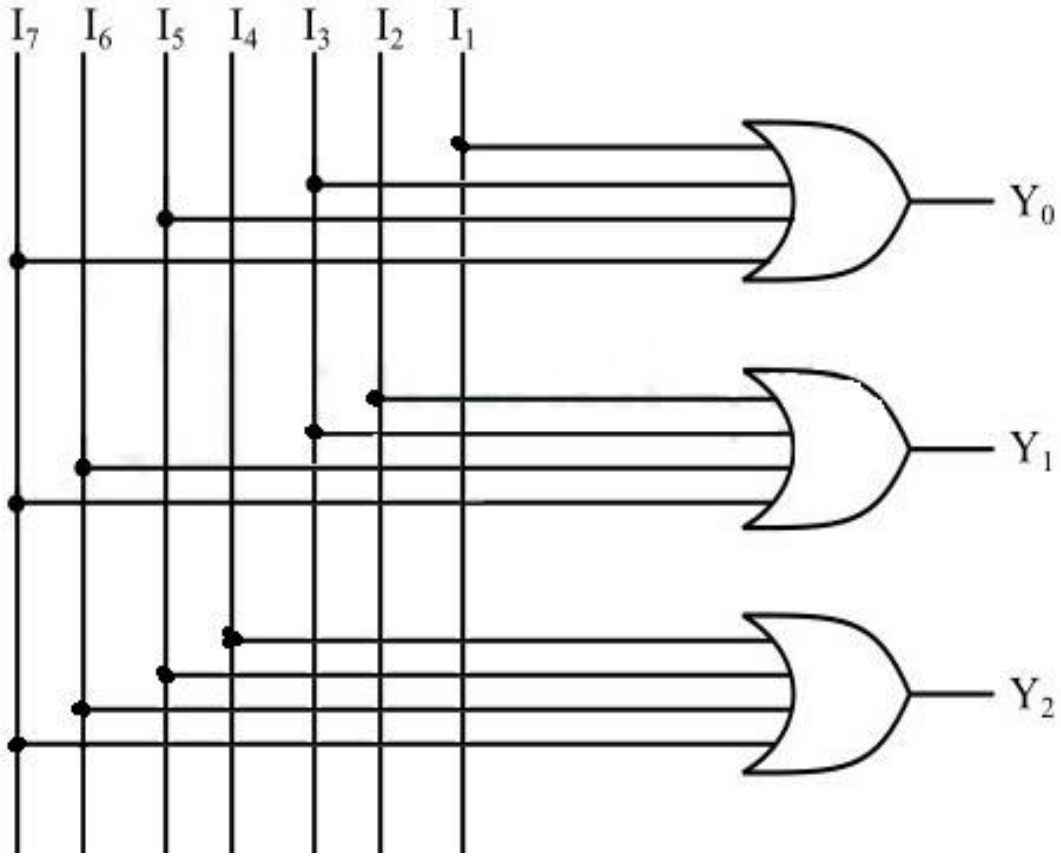
Inputs								**Outputs**		
I_7	**I_6**	**I_5**	**I_4**	**I_3**	**I_2**	**I_1**	**I_0**	**Y_2**	**Y_1**	**Y_0**
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1

0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

From this truth table, we can write the Boolean expression for the outputs of the octal to binary encoder as follows.

$$Y_0 = I_1 + I_3 + I_5 + I_7$$
$$Y_1 = I_2 + I_3 + I_6 + I_7$$
$$Y_2 = I_4 + I_5 + I_6 + I_7$$

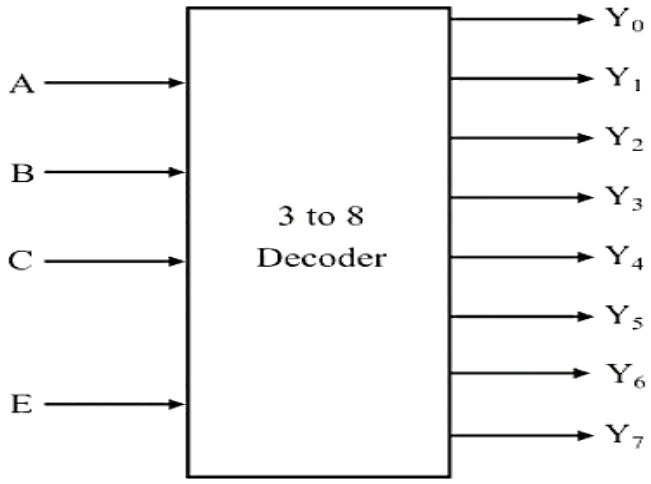
The logic circuit diagram of 8 to 3 encoder:



8) Design and explain 3x8 Decoder with its truth table and logic diagram.

Ans: The 3 to 8 decoder is one that has 3 input lines and 8 output lines.

The functional block diagram of the 3 to 8 decoder is shown in Figure:



- When this decoder is enabled with the help of enable input E, then it's one of the eight outputs will be active for each combination of inputs.
- The operation of this 3-line to 8-line decoder can be analyzed with the help of its function table which is given below.

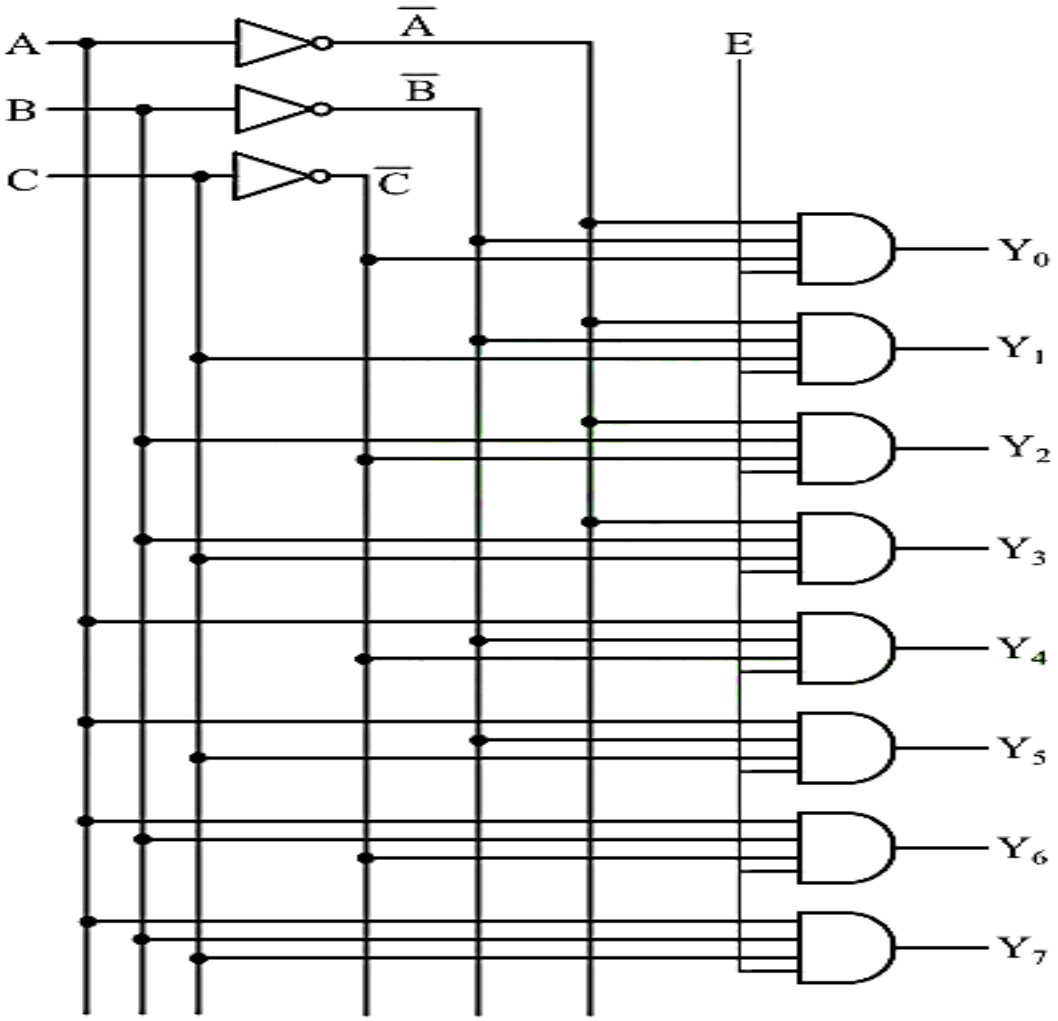
Inputs				**Outputs**							
E	**A**	**B**	**C**	**Y_7**	**Y_6**	**Y_5**	**Y_4**	**Y_3**	**Y_2**	**Y_1**	**Y_0**
0	X	X	X	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
1	0	0	1	0	0	0	0	0	0	1	0
1	0	1	0	0	0	0	0	0	1	0	0
1	0	1	1	0	0	0	0	1	0	0	0
1	1	0	0	0	0	0	1	0	0	0	0
1	1	0	1	0	0	1	0	0	0	0	0

1	1	1	0	0	1	0	0	0	0	0	0
1	1	1	1	1	0	0	0	0	0	0	0

Using this function table, we can derive the Boolean expression for each output as follows:

$$Y_0 = E\,\bar{A}\,\bar{B}\,\bar{C}$$
$$Y_1 = E\,\bar{A}\,\bar{B}\,C$$
$$Y_2 = E\,\bar{A}\,B\,\bar{C}$$
$$Y_3 = E\,\bar{A}\,B\,C$$
$$Y_4 = E\,A\,\bar{B}\,\bar{C}$$
$$Y_5 = E\,A\,\bar{B}\,C$$
$$Y_6 = E\,A\,B\,\bar{C}$$
$$Y_7 = E\,A\,B\,C$$

The logic circuit diagram of 3 to 8 decoder:


A
B
C
$\overline{A}$
$\overline{B}$
$\overline{C}$
E
Y_0
Y_1
Y_2
Y_3
Y_4
Y_5
Y_6
Y_7

9) Explain the operation of a Multiplexer. Design and explain a 4-to-1 multiplexer using logic gates, including its truth table.

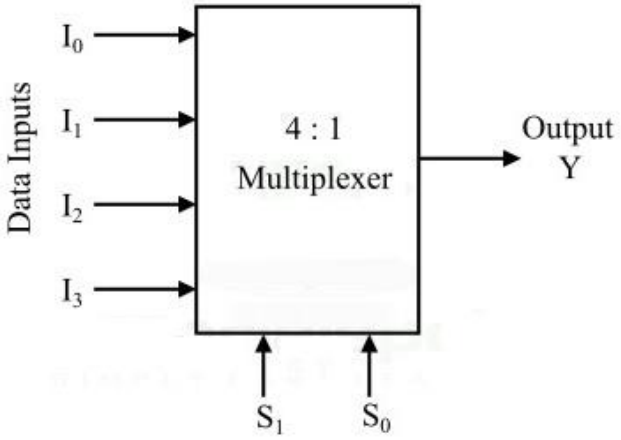
Ans:

Operation of Multiplexer:

- A multiplexer, also referred to as MUX, is a combination logic circuit that is designed to accept multiple input signals and transfer only one of them through the output line.
- In simple words, a multiplexer is a digital logic device that selects one-out-of-N (N = 2^n) input data sources and transmits the selected data to a single output line.
- The multiplexer is also called data selector as it selects one from several.

4 to 1 Multiplexer:

4×1 Multiplexer has four data inputs I_3, I_2, I_1 & I_0, two selection lines s_1 & s_0 and one output Y. The block diagram of 4×1 Multiplexer is shown in the following figure.



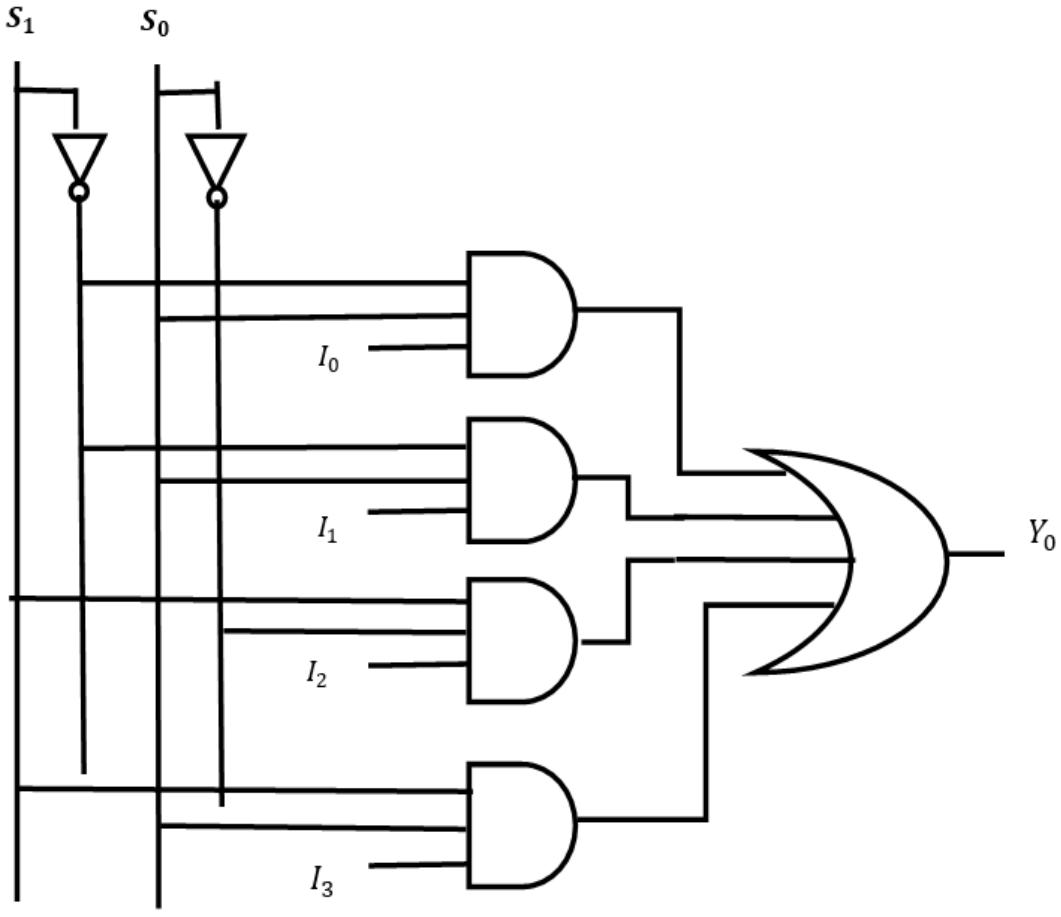
- One of these 4 inputs will be connected to the output based on the combination of inputs present at these two selection lines.
- Truth table of 4×1 Multiplexer is shown below.

Selection Lines		**Output**
S_1	**S_0**	**Y**
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

From Truth table, we can directly write the **Boolean function** for output, Y as

$$\mathbf{Y = \overline{S_1}\,\overline{S_0}I_0 + \overline{S_1}S_0I_1 + S_1\overline{S_0}I_2 + S_1S_0I_3}$$

The Logic circuit diagram of 4×1 multiplexer is shown in the following figure:



10) Explain the operation of a Demultiplexer. Design and explain a 1-to-4 demultiplexer with its truth table, logic diagram.

Ans:

Operation of Demultiplexer:

- A Demultiplexer is a combinational logic circuit that accepts a single input and distributes it over several output lines. Demultiplexer is also termed as DEMUX.
- Demultiplexer is used to transmit the same data to different destinations, hence it is also known as data distributor.
- Demultiplexer has only one data input line, 2^n output lines, and n select lines.

1 to 4 DEMULTIPLEXER:

The 1×4 Demultiplexer has 1 input line (I), 2 select line (S_0 and S_1), and 4 output lines (Y_0, Y_1, Y_2, and Y_3). The logic level applied to the select lines determines the output line to which the input data (I) will be transmitted.

- The operation of the 1×4 Demultiplexer can be understood with the help of its function table given below.

Select Line		**Outputs**			
S_1	S_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	I
0	1	0	0	I	0
1	0	0	I	0	0
1	1	I	0	0	0

From this truth table of 1×4 Demultiplexer, we can directly write the Boolean expression for each output as follow.

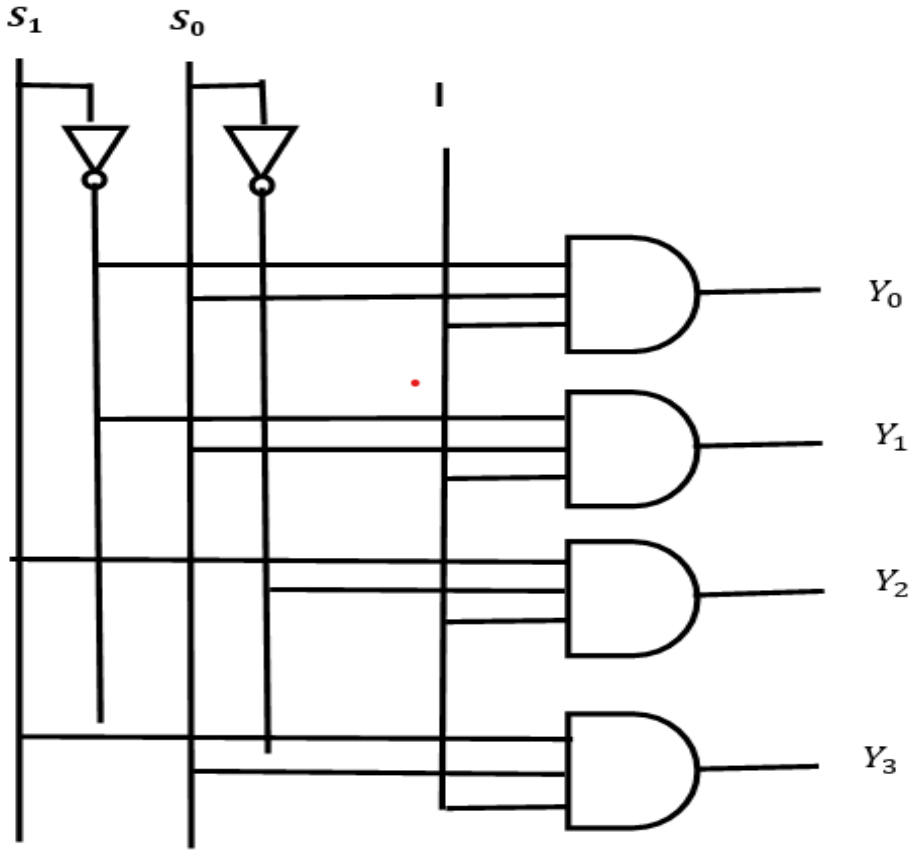
$$Y_0 = \overline{S_1}\,\overline{S_0}\,I$$

$$Y_1 = \overline{S_1}\,S_0\,I$$

$$Y_2 = S_1\,\overline{S_0}\,I$$

$$Y_3 = S_1\,S_0\,I$$

The Logic circuit diagram of 1×4 multiplexer is shown in the following figure:



MODULE-4

SEQUENTIAL LOGIC CIRCUITS

Short Questions and Answers

1) What is a latch?

A latch is a basic sequential logic circuit used to store one bit of information. It can store either 0 or 1. A latch is a level-sensitive memory device. The output remains in the stored state until the input condition changes. Latches are used in data storage and digital circuits.

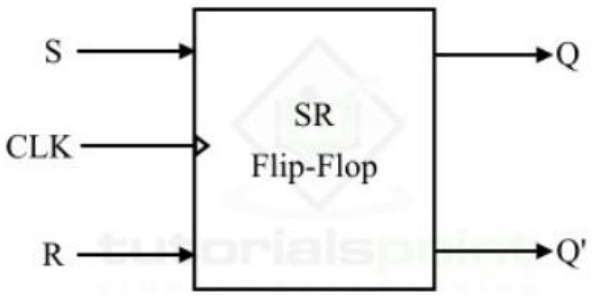
2) What is a flip-flop?

A flip-flop is a sequential logic circuit used to store one bit of binary data. It has two stable states, 0 and 1. A flip-flop is an edge sensitive memory device and it normally changes its state according to a clock signal. It is a basic building block of counters and registers.

Examples: are S-R, D, T and J-K flip-flops.

3) What is a clocked S-R flip-flop?

A clocked S-R flip-flop is a sequential circuit having Set (S), Reset (R), and Clock inputs. The output changes only when the required clock signal is applied.

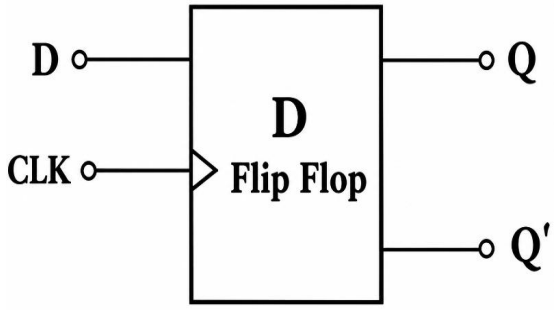


clk	S	R	Q_{n+1}	State
0	X	X	No change	No change
1	0	0	Qn	No change
1	0	1	0	Reset
1	1	0	1	Set
1	1	1	X	Invalid

- When S = 1 and R = 0, the output is set to 1.
- When S = 0 and R = 1, the output is reset to 0.

4) What is a D flip-flop?

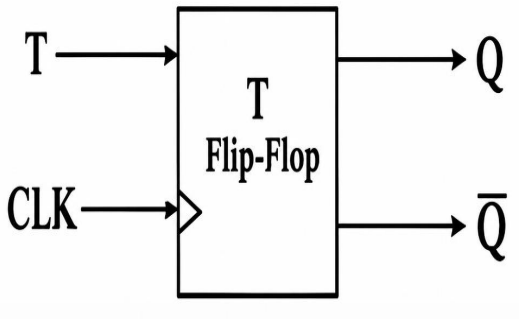
A D flip-flop is also called a Data or Delay flip-flop. It has one main input called D (Data) and a clock input. The value of D is transferred to the output when the clock is applied. It is used for data storage and data transfer. It eliminates the invalid condition found in the S-R flip-flop.



D	Q_{n+1}	State
0	0	Reset
1	1	Set

5) What is a T flip-flop?

A T flip-flop is also called a Toggle flip-flop. It has one input called T and a clock input.

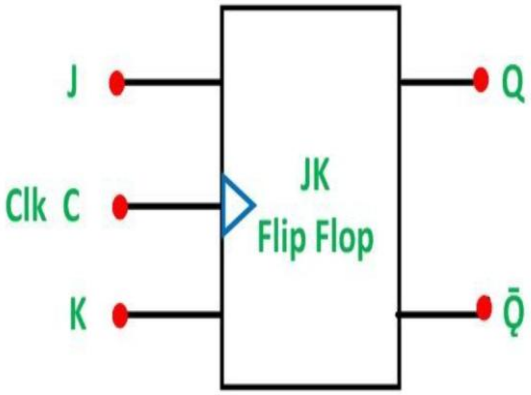


T	Q_{n+1}	State
0	Q_n	No Change
1	Q_n'	Toggle

- When T = 0, the output remains unchanged.
- When T = 1, the output changes to its opposite state.
- It is commonly used in counters and frequency division circuits.

6) What is a J-K flip-flop?

A J-K flip-flop is a modified version of the S-R flip-flop. It has two inputs called J and K, along with a clock input. It can perform hold, set, reset, and toggle operations.



J	K	Q_{n+1}	State
0	0	Qn	No change
0	1	0	Reset
1	0	1	Set
1	1	Qn’	Toggle

- There is no invalid condition when J = K = 1.
- It is widely used in counters and sequential circuits.

7) What is a master-slave J-K flip-flop?

A Master-Slave J-K flip-flop consists of two J-K flip-flops, called Master and Slave. The Master receives the input during one clock level, and the Slave receives the result during the opposite clock level. It allows the output to change only once during a clock cycle. It is mainly used to avoid the race-around condition. It is useful in counters and sequential circuits.

8) What is a counter?

A counter is a sequential logic circuit used to count the number of clock pulses. It is constructed using a group of flip-flops. The output of the counter changes according to the applied clock pulses. Counters can be classified as synchronous and asynchronous counters. They are used in digital clocks, timers and frequency counters.

9) What is a ring counter?

A ring counter is a special type of shift-register counter. It consists of a group of flip-flops connected in a circular form. The output of the last flip-flop is connected to the input of the first flip-flop. A single 1 or 0 circulates through the flip-flops with each clock pulse. It is used in sequence generation and control circuits.

10) What is a shift register?

A shift register is a group of flip-flops connected together to store binary data. It can move or shift data from one flip-flop to another. Data can be shifted either to the left or right. Shift registers are used for data storage and data transfer. They are also used for serial-to-parallel and parallel-to-serial conversion.

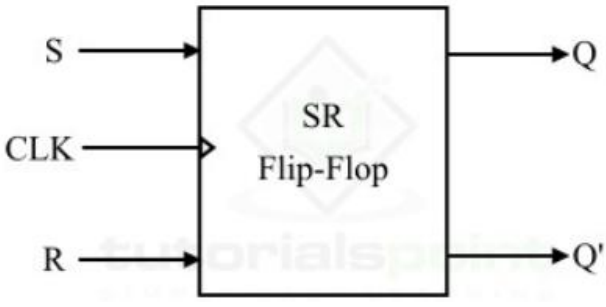
Long Question and Answers

1) Explain the working of an S-R flip-flop with a neat diagram and truth table?

Ans: It is a Flip Flop with two inputs, one is S and the other is R. **S** here stands for Set and **R** here stands for Reset. Set basically indicates set the flip flop which means output 1 and reset indicates resetting the flip flop which means output 0. Here, a clock pulse is supplied to operate this flip-flop, hence it is a clocked flip-flop.

Basic Block Diagram of SR Flip Flop:

The basic block diagram contains **S** and **R** inputs, and between them is clock pulse, **Q** and **Q'** is the complemented outputs.



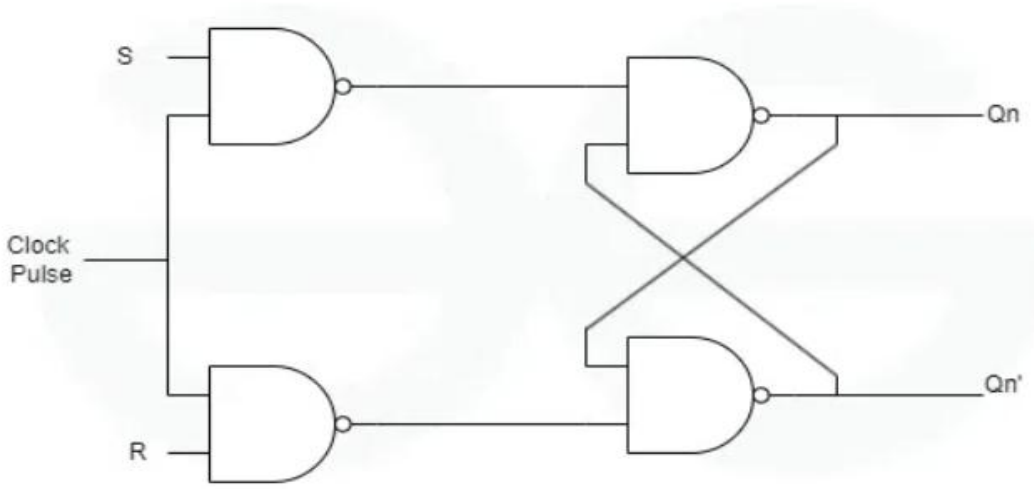
SR Flip Flop basic Block diagram

Working of SR Flip Flop:

- **Case 1** : Let's say, **S=0** and **R=0** , then output of both NAND gates will be 0 and the value of Q and Q' will be same as their previous value, i.e, No change condition (or) Hold state.

- **Case 2** : Let's say, **S=0 and R=1** , then output of both NAND gates will be 1 and 0, correspondingly the value of Q will be 0 as one of input is 1 and it is a NOR gate so it will ultimately gives 0, hence Q gets 0 value, similarly Q' will be 1.
- **Case 3** : Let's say, **S=1 and R=0** , then output of both NAND gates will be 0 and 1, correspondingly the value of Q' will be 0 as one of input to NOR gate is 1, so output will be 0 ultimately and this 0 value will go as input to upper NOR gate, and hence Q will become 1.
- **Case 4** : Let's say, **S=1 and R=1** , then output of both NAND gates will be 1 and 1 which is invalid, as the outputs should be complement of each other.

SR Flip Flop Construction using **4 NAND Gates**



Truth Table of SR Flip Flop

Given Below is the Truth Table of SR Flip Flop

S	R	Q_{n+1}	State
0	0	Qn	No change
0	1	0	Reset
1	0	1	Set
1	1	X	Invalid

2) Explain the working of a D flip-flop with a neat diagram and truth table?

Flip Flop is an electronic device or to be precise a kind of memory component that can hold one bit of data. A flip flop has two states, that is "SET" and "RESET". Those states are represented with the binary values 0 and 1. The flip flop remains in its current state until its receives a signal that switches it to opposite state. A clock or pulse signal may "trigger" the flip flop to change state.

D Flip Flop

D flip flop is an electronic device that is known as "delay flip flop" or "data flip flop" which is used to store single bit of data.

- The D flip flop has two inputs, data and clock input which controls the flip flop.
- When clock input is high, the data is transferred to the output of the flip flop.
- When the clock input is low, the output of the flip flop is held in its previous state.

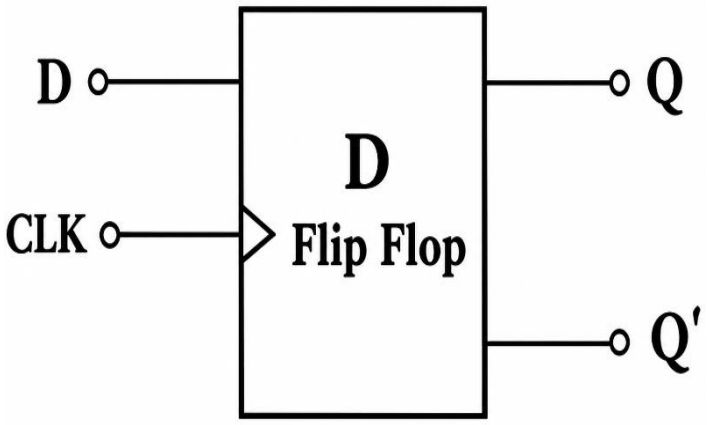


Fig: D Flip Flop basic Block diagram

Working of D Flip Flop:

D flip flop consist of a single input D and two outputs (Q and Q'). The basic working of D Flip Flop is as follows:

- When the clock signal is low, the flip flop holds its current state and ignores the D input.
- When the clock signal is high, the flip flop samples and stores D input.

- The value that was previously fed into the D input is reflected at the flip flop's Q output.
  - If D = 0 then Q will be 0.
  - If D = 1 then Q will be 1.
- The Q' output of the flip flop is complemented by the Q output.
  - If Q = 0 then Q' will be 1.
  - If Q = 1 then Q' will be 0.

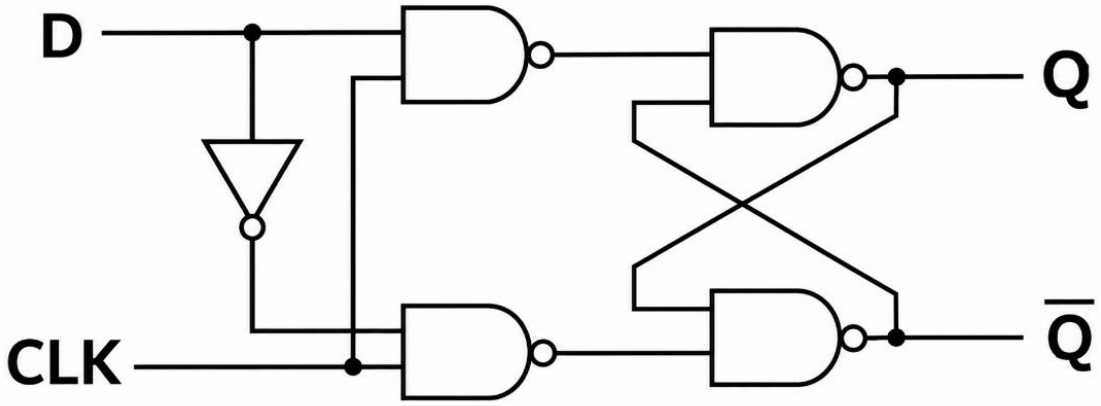


Fig: Logic Circuit of D Flip Flop

Truth Table of D Flip Flop

Given Below is the Truth Table of D Flip Flop

D	Q_{n+1}	State
0	0	Reset
1	1	Set

3) Explain the working of a T flip-flop with a neat diagram and truth table?

Ans: T flip flop is known as **Toggle Flip Flop** because it is able to toggle its output depending upon on the input. Toggle basically indicates that the bit will be flipped i.e., either from 1 to 0 or from 0 to 1. Here, a clock pulse is supplied to operate this flip flop, hence it is a clocked flip-flop.

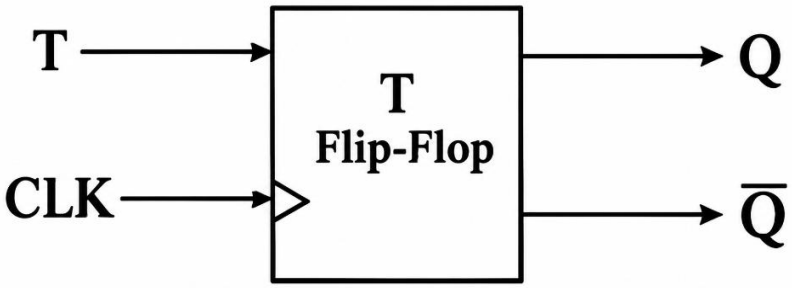


Fig: T Flip Flop basic Block diagram

Working of T Flip Flop

When **T = 0**, the circuit remains in its previous state. The feedback maintains the existing output.

$$T = 0 \Rightarrow Q_{next} = Q$$

Therefore, there is **no change in the output**. This is called the **Hold operation**.

When **T = 1**, the circuit changes its output to the complementary state.

$$T = 1 \Rightarrow Q_{next} = \overline{Q}$$

Therefore, the output **toggles** whenever the active clock condition occurs. This is called the **Toggle operation**.

The two cross-coupled NAND gates produce complementary outputs **Q and $\overline{Q}$** and provide the required memory.

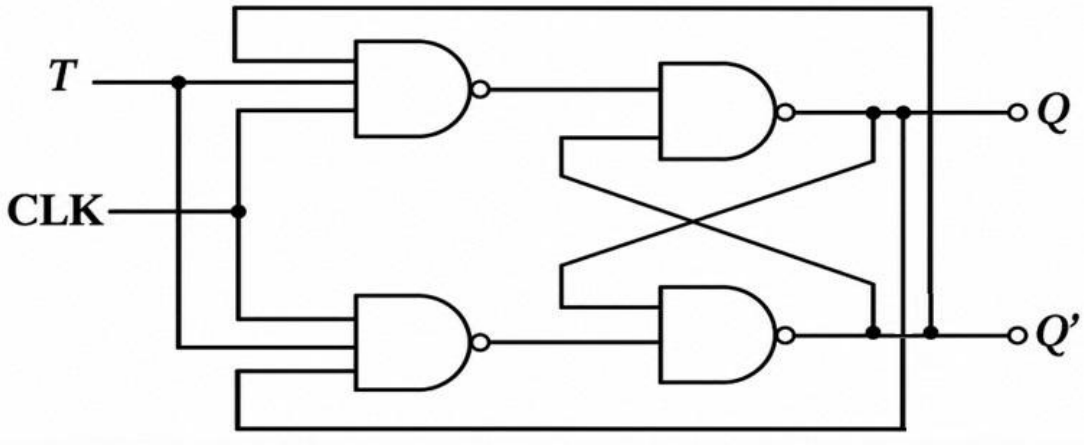


Fig: Logic Circuit of T Flip Flop

Truth Table of T Flip Flop

Given Below is the Truth Table of T Flip Flop

T	Q_{n+1}	State
0	Q_n	No change
1	Q_n'	Toggle

4) Explain the working of a J-K flip-flop with a neat diagram and truth table?

It is one kind of sequential logic circuit which stores binary information in bitwise manner. It consists of two inputs and two outputs. Inputs are Set(J) & Reset(K) and their corresponding outputs are Q and Q'.

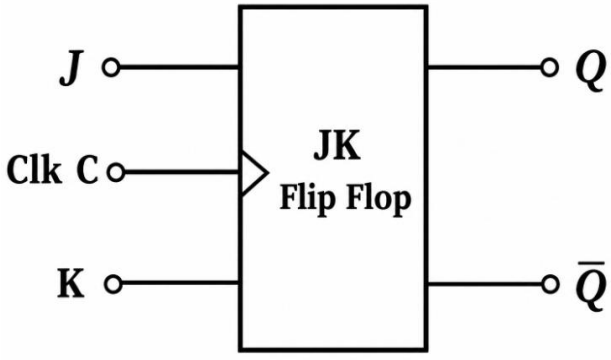


Fig: JK Flip Flop basic Block diagram

1. J = 0, K = 0 (No Change): Here, the output Q(n+1) stays the same. This means the next state (Q(n+1)) is just like the current one (Qn).
2. J = 0, K = 1 (Reset State): In this case, the next state is reset to 0 (Q(n+1) = 0), no matter what the current state is (Qn).
3. J = 1, K = 0 (Set State): Now, the next state gets set to 1 (Q(n+1) = 1), again, no matter what's happening in the current state (Qn).
4. J = 1, K = 1 (Toggle State): In this state, the output Q(n+1) toggles. So if the current state is set (Qn = 1), it will change to 0 (Q(n+1) = 0). If it's reset (Qn = 0), then it flips to 1 (Q(n+1) = 1).

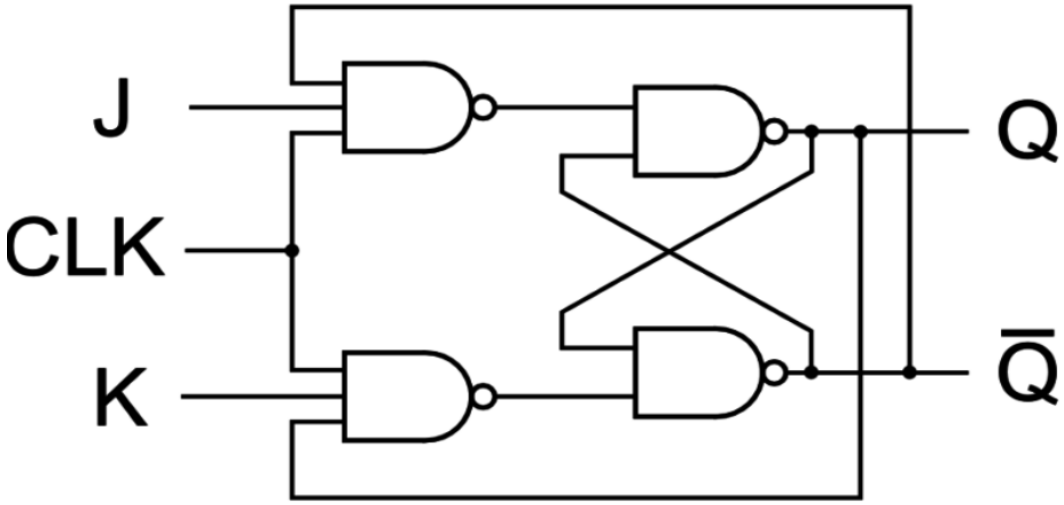


Fig: Logic Circuit of JK Flip Flop

Truth Table of JK Flip Flop

Given Below is the Truth Table of JK Flip Flop

J	K	Q_{n+1}	State
0	0	Qn	No change
0	1	0	Reset
1	0	1	Set
1	1	Qn'	Toggle

5) Explain the working of a Master-Slave J-K flip-flop with a neat diagram?

Ans: Master-Slave JK Flip Flop:

- When J = K = 1 and the clock stays high, a JK flip-flop output toggles continuously, causing the race around condition. Using short clock pulses led to the Master-Slave JK Flip-Flop to ensure stable operation.
- The Master-Slave JK Flip-Flop consists of two JK flip-flops in series, with one as the master and the other as the slave. The master output feeds the slave input, and the slave output is fed back to the master. An inverter provides the inverted clock to the slave, so only one flip-flop is active at a time. This ensures stable and predictable output.

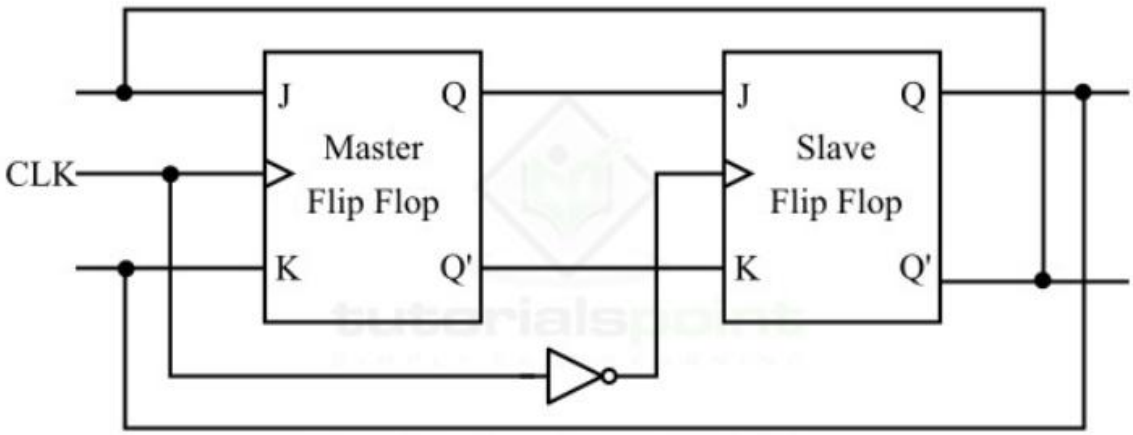


Fig: Block Diagram of Master Slave JK Flipflop

- The Master-Slave JK Flip-Flop is made of two JK flip-flops in series.
- One flip-flop acts as the master, the other as the slave.
- Master output → Slave inputs, and slave output → Master inputs.
- An inverter provides the inverted clock pulse to the slave.

1. If CP = 1 for master, CP = 0 for slave.
2. If CP = 0 for master, CP = 1 for slave.

This configuration ensures only one flip-flop is active at a time, providing stable output.

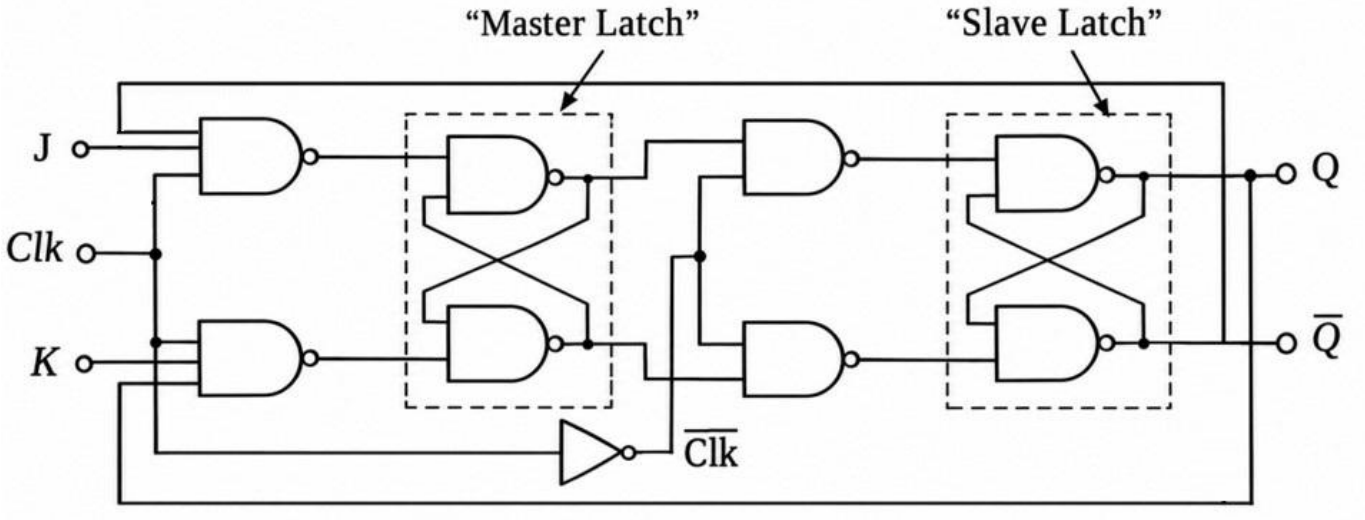


Fig: Master Slave Flip Flop

Working of a master slave flip flop :

1. When the clock pulse goes to 1, the slave is isolated; J and K inputs may affect the state of the system. The slave flip-flop is isolated until the CP goes to 0. When the CP goes back to 0, information is passed from the master flip-flop to the slave and output is obtained.
2. Firstly the master flip flop is positive level triggered and the slave flip flop is negative level triggered, so the master responds before the slave.
3. If J=0 and K=1, the high Q' output of the master goes to the K input of the slave and the clock forces the slave to reset, thus the slave copies the master.
4. If J=1 and K=0, the high Q output of the master goes to the J input of the slave and the Negative transition of the clock sets the slave, copying the master.
5. If J=1 and K=1, it toggles on the positive transition of the clock and thus the slave toggles on the negative transition of the clock.
6. If J=0 and K=0, the flip flop is disabled and Q remains unchanged.

6) Write the differences between synchronous and asynchronous counters.

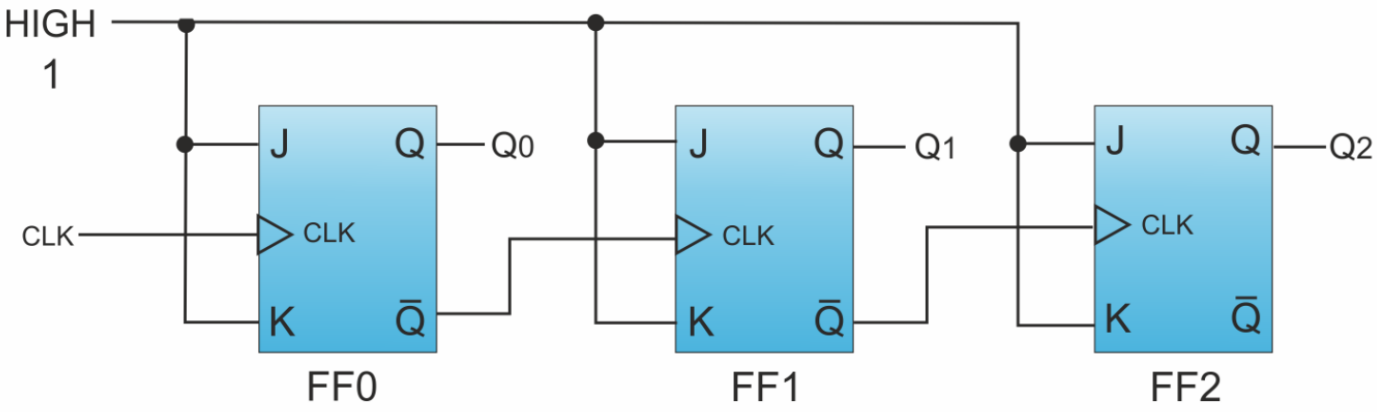
Synchronous Counter	**Asynchronous Counter**
1. All flip-flops get the same clock.	1. Flip-flops get the clock one after another.
2. Flip-flops change at the same time.	2. Flip-flops change one by one.
3. It is faster.	3. It is slower.
4. It has less delay.	4. It has more delay.
5. Circuit is more complex.	5. Circuit is simple.
6. All flip-flops use a common clock.	6. Only the first flip-flop gets the external clock.
7. Used in high-speed circuits.	7. Used in low-speed circuits.
8. Also called a parallel counter.	8. Also called a ripple counter.
9. Output changes quickly.	9. Output changes with delay.
10. Example: High-speed digital counter.	10. Example: Simple event counter.

7) Explain the working of a 3-bit Asynchronous counter with a neat diagram and counting sequence?

Ans: 3-Bit Asynchronous Counter:

- A 3-bit asynchronous counter, also called a ripple counter, is a sequential circuit made using three flip-flops. It counts from 000 to 111, giving a total of 8 states.
- The output of each flip-flop provides the clock for the next flip-flop. Therefore, the flip-flops do not receive the clock simultaneously, which is why it is called an asynchronous counter.

3-BIT ASYNCHRONOUS COUNTER



3-bit Asynchronous Counter Block diagram

Working

1. J = K = 1 is connected to all three JK flip-flops.
2. When J = K = 1, each JK flip-flop operates in toggle mode.
3. The external clock is applied only to the first flip-flop.
4. The complemented output of the first flip-flop is connected to the clock input of the second flip-flop.
5. The complemented output of the second flip-flop is connected to the clock input of the third flip-flop.
6. Therefore, the clock signal ripples through the flip-flops, so it is called an asynchronous/ripple counter.
7. The outputs Q_2 Q_1 Q_0 give the 3-bit binary count.

Table: Counting Sequence Table:

Clock Pulse	Q_2	Q_1	Q_0
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

Counting sequence:

000 → 001 → 010 → 011 → 100 → 101 → 110 → 111 → 000

Number of states = 2^3 = 8, so it is a MOD-8 counter.

8) Explain the working of a ring counter with a neat diagram and counting sequence?

Ans: A ring counter is a typical application of the Shift register. The ring counter is almost the same as the shift counter. The only change is that the output of the last flip-flop is connected to the input of the first flip-flop in the case of the ring counter but in the case of the shift register it is taken as output. Except for this, all the other things are the same.

Ring Counter:

It is a type of digital counter used in circuits, typically built with flip-flops. It works by shifting a 1 through a series of flip-flops, one at a time, in a loop. It's called a ring counter because the 1 loops back to the start once it reaches the end, forming a cycle.

No. of states in Ring counter = No. of flip-flop used

Working Of Ring Counter

- A Ring Counter is typically built using flip-flops (D, T, or JK flip-flops).
- It consists of n flip-flops, where n is the number of states in the counter. The number of flip-flops determines the number of unique states the counter will cycle through.
- One bit in the counter is set to 1, and the rest of the bits are set to 0. In each clock cycle, the 1 bit shifts through the flip-flops, and the pattern repeats.
- The output of the last flip-flop is fed back into the first flip-flop, forming a loop, hence the name Ring Counter.

Example of a 4-bit Ring Counter

Consider a 4-bit Ring Counter:

Initial state: 1000 (the first flip-flop is set to 1, and others are set to 0).

Clock cycle 1: The '1' bit shifts to the next flip-flop, resulting in the state 0100.

Clock cycle 2: The '1' bit shifts to the next flip-flop, resulting in the state 0010.

Clock cycle 3: The '1' bit shifts to the next flip-flop, resulting in the state 0001.

Clock cycle 4: The '1' bit shifts back to the first flip-flop, and the cycle repeats, resulting in the state 1000.

State Sequence:

1000 → 0100 → 0010 → 0001 → 1000

So, for designing a 4-bit Ring counter we need 4 flip-flops as designed below :

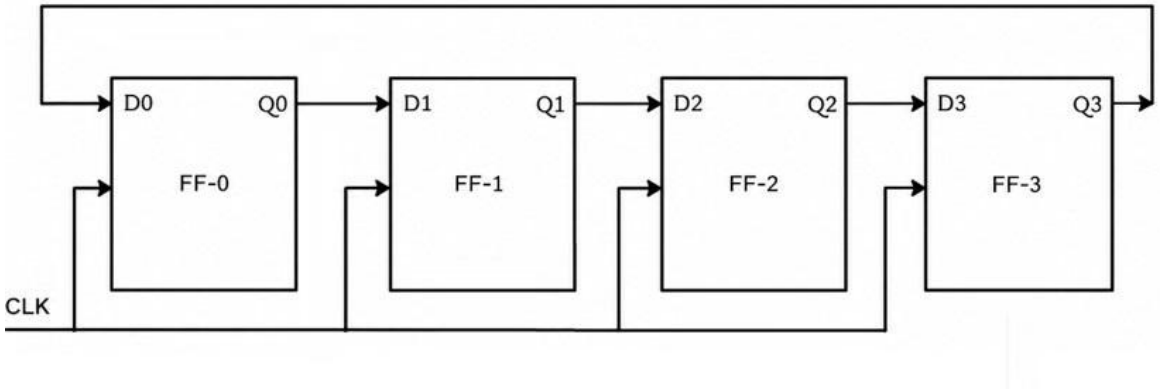


Fig: 4bit Ring Counter Block Diagram

Clock	Q_0	Q_1	Q_2	Q_3
0	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0

Table: 4bit Ring Counter Truth Table

9) Explain left-shift registers with suitable diagrams and examples?

Ans: Shift Register:

A **shift register** is a group of flip-flops used to store and shift binary data. Each flip-flop stores one bit of data. The data is shifted from one flip-flop to another with every clock pulse.

Left-Shift Register:

In a left-shift register, the data moves one position to the left for every clock pulse. A new bit enters from the right side**.**

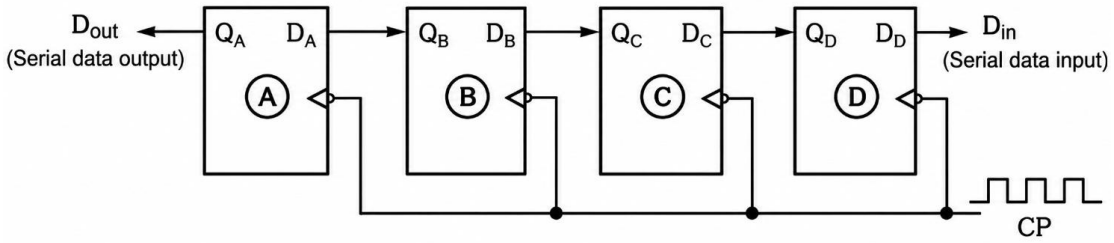


Fig: Block Diagram of left Shift

Working:

- All flip-flops receive the same clock pulse.
- At each clock pulse, the stored bits shift one position to the **left**.
- A new bit enters from the **right**.
- The leftmost bit is shifted out as the serial output.

We will illustrate the entry of the four bit binary number 1111 into the register, beginning with the left-most bit. Initially, register is cleared.

So $Q_AQ_BQ_CQ_D$ = 0000

CP	Q_3	Q_2	Q_1	Q_0	D_{in}
Initially	0	0	0	0	1
↓ 1st	0	0	0	1	1
↓ 2nd	0	0	1	1	1
↓ 3rd	0	1	1	1	1
↓ 4th	1	1	1	1	1

Table: The shift left operation.

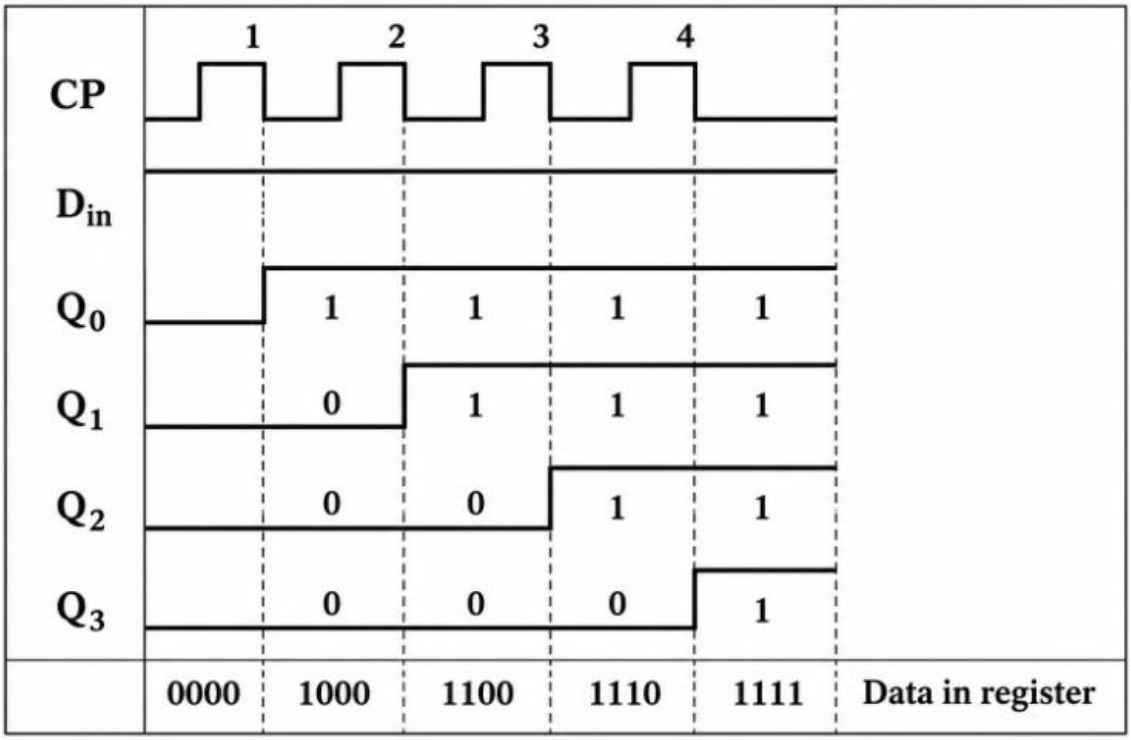


Fig: wave forms for shift left operation

10.) Explain right-shift registers with suitable diagrams and examples?

Ans: In a right-shift register, the data moves one position to the right for every clock pulse. A new bit enters from the left side.

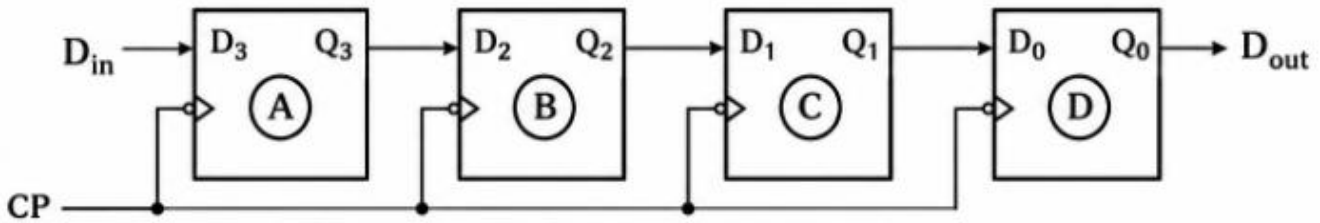


Fig: Block Diagram of right Shift

Working

- All flip-flops receive the same clock pulse.
- At each clock pulse, the stored bits shift one position to the **right**.
- A new bit enters from the **left**.
- The rightmost bit is shifted out as the serial output.

We will illustrate the entry of the four bit binary number 1111 into the register, beginning with the left-most bit. Initially, register is cleared.

So $Q_3Q_2Q_1Q_0$= 0000

CP	D_{in}	Q_3	Q_2	Q_1	Q_0
Initially	1	0	0	0	0
↓ 1st	1	1	0	0	0
↓ 2nd	1	1	1	0	0
↓ 3rd	1	1	1	1	0
↓ 4th	1	1	1	1	1
Data in register	0000	1000	1100	1110	1111

Table: The shift right operation

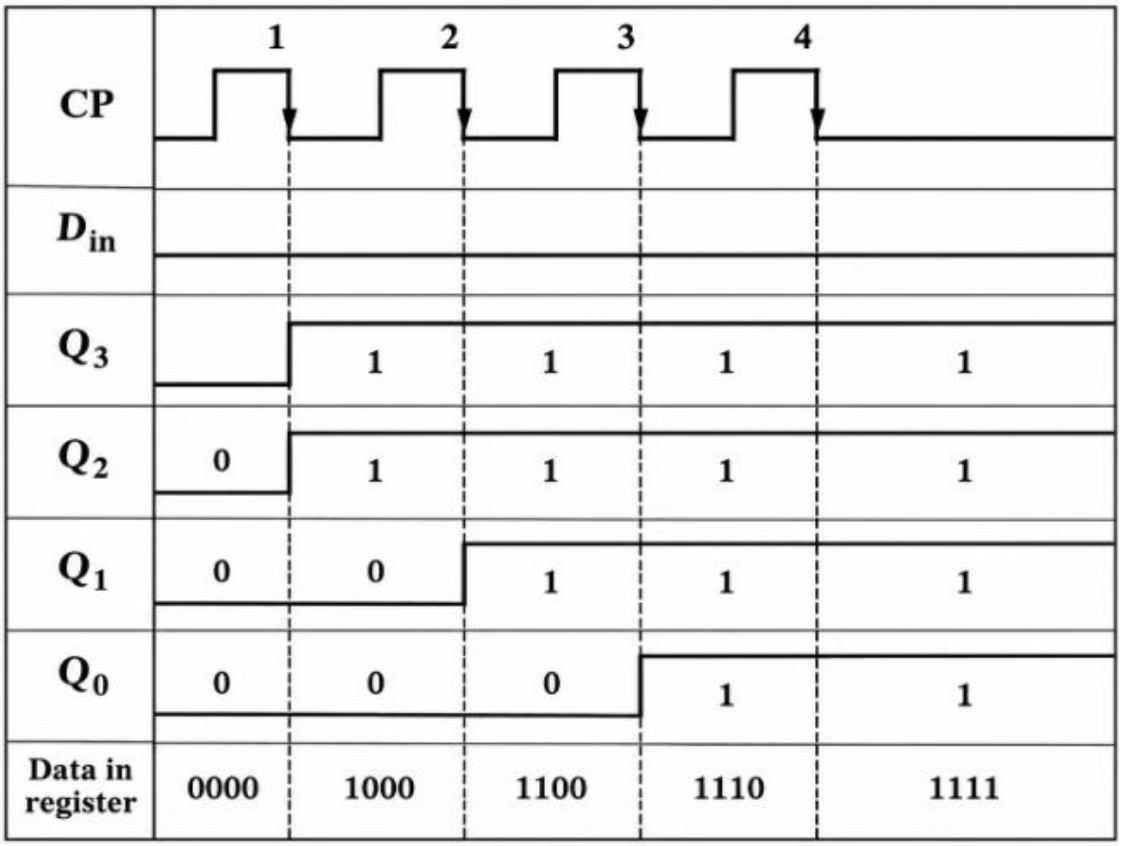


Fig: wave forms for shift right operation

MODULE-5

MEMORY DEVICES

Short Question and Answers

1) What is a memory device?

Ans: A memory device in digital electronics is a hardware component built from electronic circuits that stores binary data (0 s and 1s) and instructions for processing.

Classification of Memory Devices:

1. Primary (Internal) Memory
2. Secondary (External) Memory

2) What is RAM?

Ans: RAM (Random Access Memory) is also known as Volatile memory (loses data when power turns off) that allows fast read and write operations for running programs.

- SRAM (Static RAM)
- DRAM (Dynamic RAM)

3) What is ROM?

Ans: Read-Only Memory (ROM) in digital electronics is a type of non-volatile memory that permanently stores data and instructions, retaining them even when the power is turned off.

- MROM
- PROM
- EPROM
- EEPROM

4) What is PROM?

Ans: PROM stands for Programmable Read-Only Memory. It is a memory chip on which information can be composed as it were once. Once data is written onto a PROM chip it cannot be modified and erased. This makes PROM more useful for applications where permanent storage of data is required.

5) What is EPROM?

Ans: An EPROM (Erasable Programmable Read-Only Memory) is a type of non-volatile memory chip that retains its data even when the power is turned off, and it can be erased and reprogrammed multiple times. This is done by exposing the chip to ultraviolet light, which causes the bits to return to their original state. EPROM is more expensive than PROM, but it is also more reliable and has a longer lifespan.

6) What is EEPROM?

Ans: EEPROM stands for Electrically Erasable Programmable Read-Only Memory. EEPROM is a type of non-volatile primary memory. EEPROM is a type of ROM that can be erased and reprogrammed electrically. This is done by applying a high voltage to the desired bits, which causes them to change from 0s to 1s. EEPROM is the most expensive type of ROM, but it is also the most reliable and has the longest lifespan.

7) What is EAPROM?

Ans: In digital electronics, EAPROM stands for Electrically Alterable Programmable Read-Only Memory. It is a specialized type of non-volatile semiconductor memory that can retain stored data even when the power supply is completely turned off.

8) What is PLA?

Ans: A Programmable Logic Array (PLA) is a type of programmable logic device used to build custom combinational and sequential digital circuits. It features a programmable AND gate array followed by a programmable OR gate array.

9) What is PAL?

Ans: Programmable Array Logic (PAL) in digital electronics is a type of programmable logic device featuring a programmable AND array followed by a fixed OR array.

- **Programmable AND Array:** Inputs pass through a matrix of AND gates with fusible links that can be programmed (blown or left intact) to create specific product terms.
- **Fixed OR Array:** The outputs of the AND gates feed into a set of permanent, non-programmable OR gates that combine the product terms into the final Sum-of-Products (SOP) expression.

10) What is the difference between RAM and ROM?

RAM	ROM
Random Access Memory	Read only Memory
Volatile Memory	Non-Volatile Memory
Data is lost when power is off	Data is remains even when power is off
Temporary storage for running programs	Permanent storage of system instructions
Can both read and write	Only can be read cannot be easily changed or overwritten
Used by CPU for processing tasks	Stores BIOS
Examples: Running app,files	Examples: Boot-up instructions, system BIOS

Long Question and Answers

1) Explain RAM and its basic working with a neat diagram?

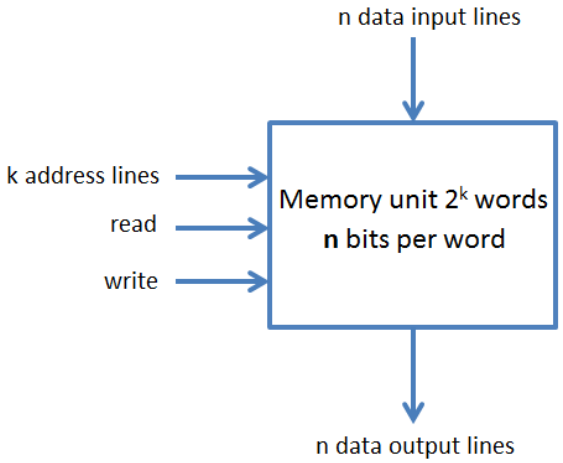
Ans: Random Access Memory (RAM) is a volatile memory in which both read and write operations can be performed. Located on the motherboard, it directly supports the CPU, allowing programs to run smoothly and information to be processed quickly.

- As volatile memory, data in RAM is lost when the computer is powered off.
- Its high speed greatly influences overall system performance, and insufficient RAM can slow program loading and reduce efficiency.
- RAM is made of tiny transistors and capacitors that store electrical charges representing data bits. It acts as the computer's short-term memory, temporarily holding data the CPU needs to access quickly.

Block Diagram of RAM:

- In Random Access Memory (RAM), the memory cells can be accessed for information transfer from any desired random location.

- That is the process of locating a word in memory is the same and requires an equal amount of time no matter where the cells are located physically in memory: thus the name "Random Access".
- Communication between memory and its environment achieved through data input and output lines, address selection lines, and control lines that specify the direction of transfer.
- A block diagram of a RAM unit is shown below figure:



The K Address ***lines*** provide the information to be stored in memory, and the ***n data output lines*** supply the information coming out of particular word chosen among the 2^k available inside the memory. The two control inputs specify the direction of transfer desired.

OPERATIONS of RAM:

- ✓ ***Write Operation***
- ✓ ***Read Operation***

The **write and read operations** are the **two operations** that a **random access memory** can **perform.**

A **Transfer-In Operation** is specified by the **Write Signal**, while a **Transfer-Out Operation** is specified by the **Read Signal.** When one of these **control signals** is **accepted,** the **memory's internal circuits perform the required function**.

The following are the steps that must be followed in order for a new word to be stored in memory:

1) Apply/Fill in the address lines with the binary address of the desired word.
2) Apply/Fill the data input lines with the data bits that must be stored in memory.
3) Activate the Write Input.

The memory device will then take the bits currently available on the input data lines and store them in the address lines' designated locations. The following are the steps that must be followed in order to transfer a stored word from memory:

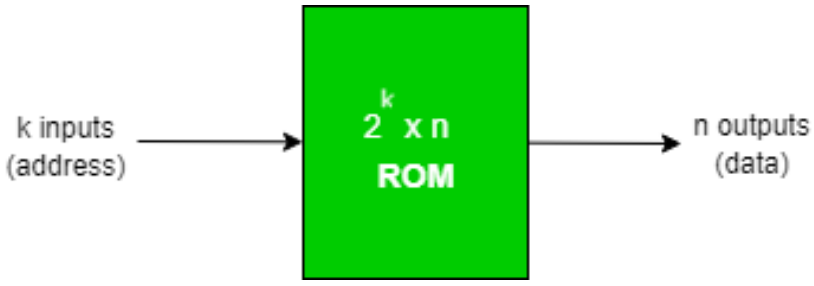
1. Apply/Fill in the address lines with the binary address of the desired expression.
2. Activate the Read Input.

The memory device will then apply the bits from the address-selected word to the output data lines. After reading, the selected word's content remains unchanged.

2) Explain ROM and its basic working with a neat diagram?

Ans: Read-Only Memory (ROM) in digital electronics is a type of non-volatile memory that permanently stores data and instructions, retaining them even when the power is turned off.

Architecture of ROM:



Information in the form of binary is stored permanently inside ROM by the manufacture, the information is injected in the form of bits. ROM consists of logic gates only, arranged in a way that they store specified bits.

Block structure:

- The unit consists of k input lines and n out lines.
- The k input lines take the input address from where we want to access the content of the ROM.
- Since the input lines are either 0 or 1 (binary form). The input lines can be referred to as 2^k total addresses and each of these addresses contains n bit of

information, which will be given out as the output of the ROM. This is specified as 2^k x n ROM.

Internal structure:

The internal structure consists of two components: the decoder and OR (logic) gates.

- The decoder is a combinational circuit. It is used to decode any encoded form like binary to understandable forms like decimal form. Within the ROM structure, the input into a decoder will be binary and the output will be represented in decimal form.
- All the OR logic gates will have outputs of the decoder as their input.

Advantages of ROM

- It is non-volatile, meaning data which was set by the manufacture will function as expected when device is turned on.
- Due to them being static, they don't need a refreshing time.
- In comparison to RAM, the circuitry is simpler.
- Data can be stored permanently.

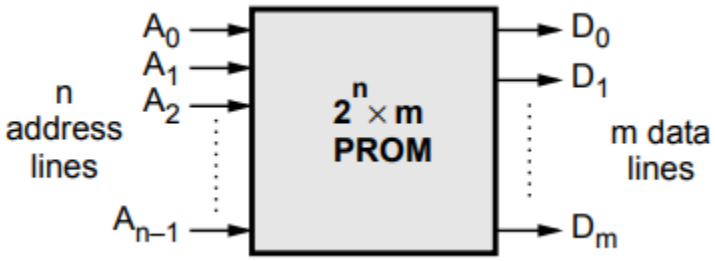
Disadvantages of ROM:

- ROM is a read only memory unit, so it can't be modified.
- If any changes are required, it's not possible.

3) Explain PROM and how it is programmed?

- PROM (Programmable Read-Only Memory) is a type of memory chip that can be programmed only once. Once data is written to a PROM, it cannot normally be erased or changed.
- Unlike RAM, PROM keeps its stored data even when the computer is switched off. A PROM is manufactured as blank memory, and a special device called a PROM programmer/burner is used to write data to it.
- PROMs are commonly used to store firmware or low-level device drivers for specific hardware.

Block diagram of PROM:



- It consists of n-input lines and m-output lines. Each bit combination of the input variables is called an address. Each bit combination that comes out of the output lines is called a word.
- The number of bits per word is equal to the number of output lines, m. The address specified in binary number denotes one of the minterms of n variables.
- The number of distinct addresses possible with n-input variables is 2n. An output word can be selected by a unique address and since there are 2n distinct addresses in PROM, there are 2n distinct words in the PROM.
- The word available on the output lines at any given time depends on the address value applied to the input lines.

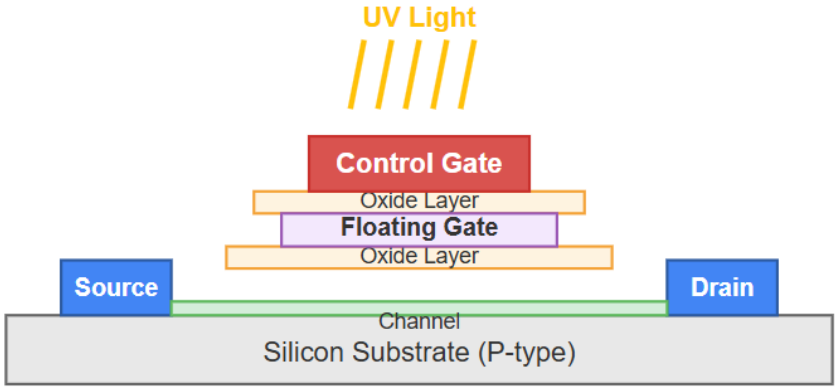
Characteristics of PROM:

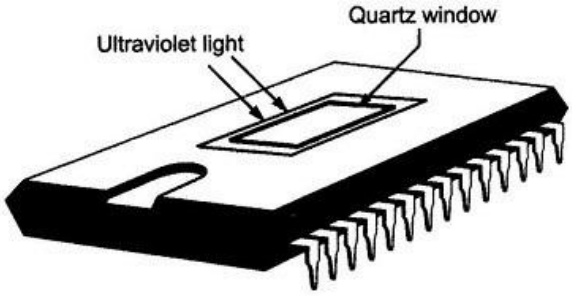
- It has programmable Random access memory.
- In prom And gates are fixed and or gates are programmable.
- Prom Works as a memory.
- It is not reusable.
- The storage endurance of PROM is high.
- Few functions can be actualized must have all the variables.

4) Explain EPROM and its working in simple steps?

- Erasable Programmable Read Only Memory (EPROM) use MOS circuitry.
- They store 1s and 0s as a packet of charge in a buried layer of the IC chip. EPROMs can be programmed by the user with a special EPROM programmer.
- The important point is that we can erase the stored data in the EPROMs by exposing the chip to ultraviolet light through its quartz window for 15 to 20 minutes.

EPROM Cell Structure





It is not possible to erase selective information, when erased the entire information is lost. The chip can be reprogrammed.

EPROM Works in Simple Steps:

1. **The Structure (Floating-Gate Transistor):**

   Inside the EPROM chip, data is stored in a grid of special transistors. Each transistor has two gates (control gate and **floating gate**) surrounded by an insulating layer of silicon oxide. The floating gate floats electrically and holds trapped electrons.

2. **Programming the Data (Writing):**

- A special device called an **EPROM programmer** or burner applies a high voltage (typically 12 to 25 volts) to the chip.
- This high voltage gives electrons enough energy to push through the insulating layer and get trapped on the floating gate.

- The presence or absence of trapped electrons changes the transistor's electrical state, representing binary data (0 or 1). Because the insulation is very strong, the data remains safely stored for 10 to 20 years without power.

3. Reading the Data

- During normal use, a lower voltage is applied to the control gate to check if the transistor conducts current.
- If electrons are trapped in the floating gate, they block the current, and the chip reads a 0.
- If the floating gate is empty, current flows freely, and the chip reads a 1.

4. Erasing the Data

- To clear the memory, the chip must be removed from the circuit board and exposed to intense **ultraviolet (UV) light** through the quartz window for 15 to 30 minutes.
- The UV photons energize the trapped electrons, causing them to pass back through the insulating layer and disperse.
- This completely resets the entire chip back to its initial state, making it ready to be reprogrammed from scratch.

5) Explain EEPROM and its working in simple steps?

Ans: EEPROM (Electrically Erasable Programmable read-only memory) is a type of non- volatile memory technology that may be programmed, erased and reprogrammed electrically.

- It allows for information to be saved even if electricity is removed, making it useful in numerous electronic devices that require information storage.
- EEPROM is normally used in microcontrollers, smart cards, and other electronic devices.

Working Principle of EEPROM:

- EEPROM works by using a grid of transistors and capacitors to save binary records.
- Each transistor and capacitor pair, called a memory cellular, can save a single bit of data.

- When a voltage is applied to the control gate of the transistor, it permits electrons to pass through the insulating layer, either charging or discharging the capacitor to represent a 1 or 0.
- To change the saved information, a better voltage is applied to the gate, which causes the insulating layer to interrupt down and erase the stored charge.
- The memory cell can then be reprogrammed with new data through applying the suitable voltages to the control gate and the other electrodes.

  There are two distinct EEPROM families: serial and parallel access. The serial access represents 90 percent of the overall EEPROM in market, where as the parallel access EEPROMs is about 10 percent.

Parallel EEPROM:

1. Parallel devices are available in higher densities more then 256bits and are generally working faster.
2. Highly Reliable and these are mostly used for military market.
3. They are pin compatible with EPROMs and flash memory devices.

Serial EEPROM:

1. Serial EEPROMs are less dense (typically from 256 bit to 256Kbit) and are slower than parallel devices.
2. They are much cheaper and used in more “commodity” applications.

6) Explain EAPROM and mention its main features?

Ans: Electrically Alterable Programmable Read-Only Memory (EAROM/EEPROM) is a non-volatile memory type that lets users change stored data using electrical signals without removing the chip from the circuit.

Main Features

- **Non-Volatile Storage:** Retains all saved data even when the power supply turns off.
- **Electrical Erasing and Programming:** Uses electrical voltages instead of ultraviolet light (required by older EPROMs) to clear and rewrite data.
- **In-Circuit Modifiability:** Allows updates while the chip remains soldered directly on the motherboard or device circuit board.

- **Byte-Level or Bit-Level Access:** Enables selective erasure and rewriting of specific bytes or small groups of bits rather than clearing the entire chip.
- **High Endurance:** Supports up to 1 million write and erase cycles in modern variants before wear-out occurs.

Key Applications in Digital Systems:

- **Microcontrollers**: Stores startup code, device IDs, and runtime logs.
- **Consumer Electronics**: Saves user preferences, volume levels, and channel setups in TVs or appliances.
- **Automotive Systems**: Holds sensor calibration data, odometer values, and engine control unit (ECU) settings.
- **IoT and Industrial Devices**: Maintains network credentials, security keys, and operational parameters.

7) Write short notes on ROM, PROM, EPROM, and EEPROM?

Ans: <u>ROM (Read-Only Memory):</u>

- **Definition:** A type of memory that is permanently programmed during the manufacturing process**.**
- **Function:** It holds essential startup instructions, such as the BIOS (Basic Input/Output System) or firmware, that a device needs to boot up.
- **Characteristics:** Data can only be read, not modified or rewritten by the user.

<u>PROM (Programmable Read-Only Memory):</u>

- **Definition**: A blank version of ROM that can be purchased and programmed by the user just once.
- **Function**: Used in embedded systems or microcontrollers where custom, permanent code is required.
- **Characteristics**: Internal fuses are blown during programming to lock the data in place; it cannot be erased or changed once set.

<u>EPROM (Erasable Programmable Read-Only Memory):</u>

- **Definition**: An upgrade to PROM that allows the stored data to be erased and rewritten.
- **Function**: Frequently used during product development and testing when firmware needs periodic updates.

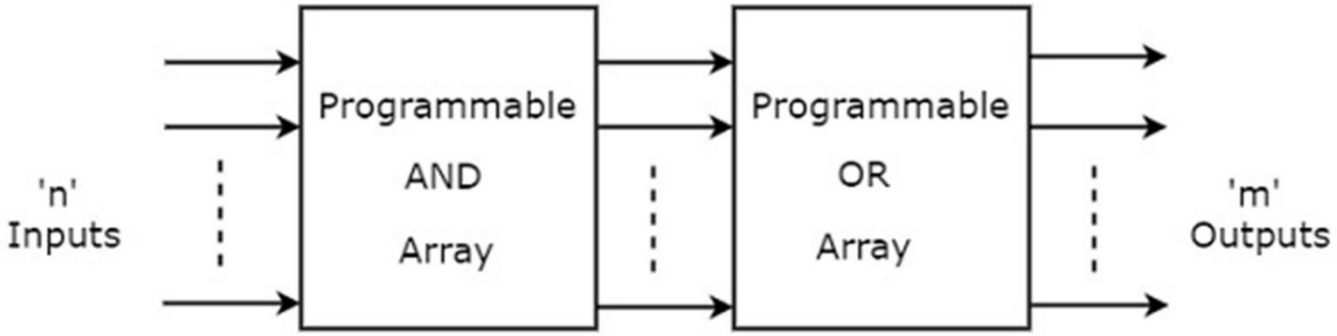
- **Characteristics**: Erasing requires removing the chip from the device and exposing its quartz crystal window to ultraviolet (UV) light for about 20 to 40 minutes.

EEPROM (Electrically Erasable Programmable Read-Only Memory):

- **Definition**: A flexible memory chip that can be erased and reprogrammed using electrical signals.
- **Function**: Used in modern computers, smart cards, and IoT devices for firmware upgrades and storing configuration settings.
- **Characteristics**: Data can be modified byte-by-byte directly inside the circuit without needing UV light or physical removal.

8) Explain the basic structure and working of a Programmable Logic Array (PLA)?

Ans: In the field of digital electronics, the PLAs are extensively used for designing combinational logic circuits. The greatest advantage of designing combinational circuits using PLAs is that PLA consist of programmable AND and programmable OR arrays which allows to implement custom desired logic functions.



Example:

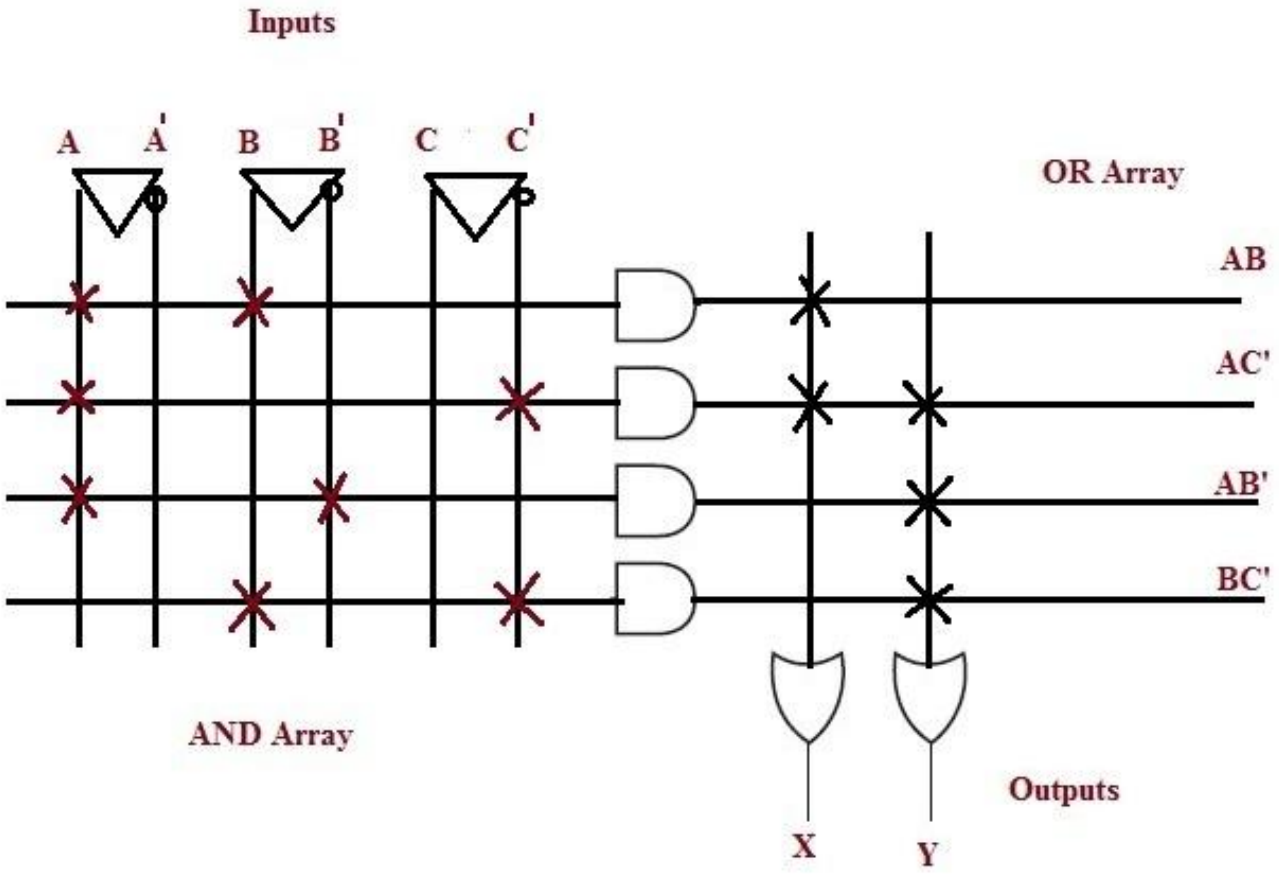
Implement the following Boolean expression with the help of programmable logic array (PLA)

X=AB+AC'

Y = AB' +BC + AC'

The above given two Boolean functions are in the form of SOP (sum of products). The product terms present in the Boolean expressions are X & Y, and one product term that is AC' is common in every equation. So, the total required logic gates for

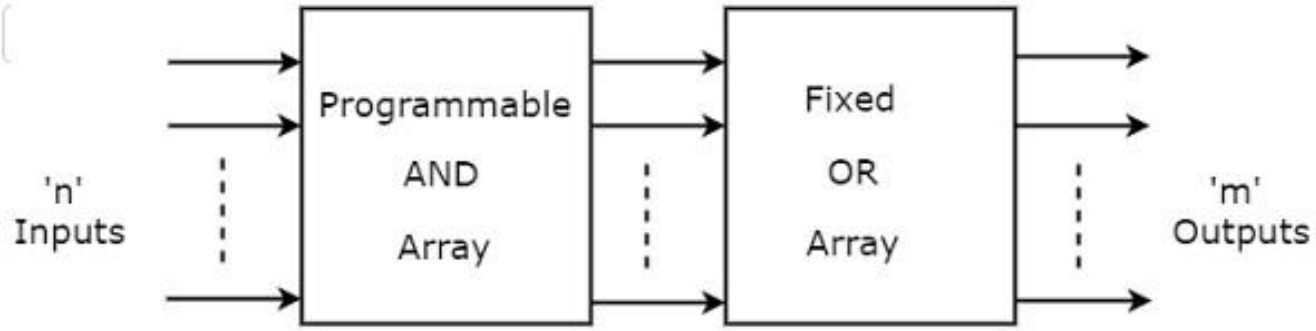
generating the above two equations is AND gates-4, OR programmable OR gates-2. The equivalent PLA logic diagram is shown below.



9) Explain the basic structure and working of Programmable Array Logic (PAL)?

Ans:

- The Programmable Array Logic (PAL) is a type of Programmable logic device used to design and implement a variety of custom logic functions.
- A PAL device consists of an array of programmable AND gates connected to a fixed array of OR gates. This array structure helps to implement various logic functions by interconnecting the input lines, AND gates and OR gates.



- We can design combinational logic circuits using Programmable Array Logic (PAL) devices.
- While designing combinational logic using PAL, it is important to note that the Boolean expression in the sum products form must be simplified to fit into each section of the PAL. Because the array of OR gates is fixed, the number of product terms fed to each OR gate cannot be changed.

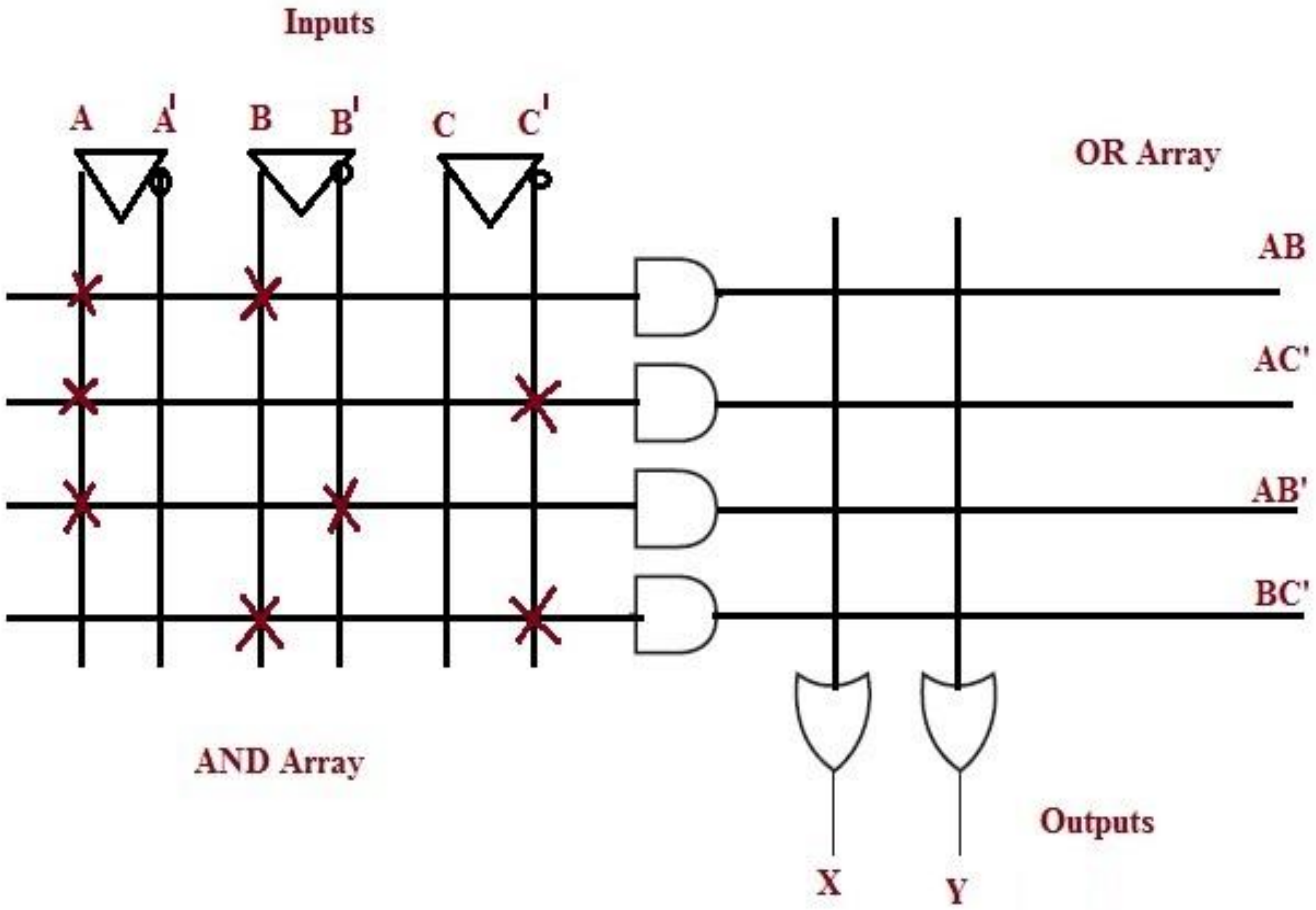
Example:

Implement the following Boolean expression with the help of Programmable array logic (PAL)

X=AB+AC'

Y= AB' + BC'

- The above given two Boolean functions are in the form of SOP (sum of products).
- The product terms present in the Boolean expressions are X & Y, and one product term that is AC' is common in every equation.
- So, the total required logic gates for generating the above two equations is AND gates-4 and OR programmable gates-2.
- The equivalent PAL logic diagram is shown below:


Inputs
A
A'
B
B'
C
C'
OR Array
AB
AC'
AB'
BC'
AND Array
Outputs
X
Y

10) Explain the differences between PLA and PAL with a simple comparison table?

Ans:

PLA (Programmable Logic Array)	**PAL (Programmable Array Logic)**
Stands for Programmable Logic Array	Stands for Programmable Array Logic
Speed is lower	Speed is higher
Has higher complexity	Has lower complexity
Supports fewer logic functions	Supports a larger number of logic functions
Cost is high	Cost is low
Less easily available	More easily available
Uses programmable AND and programmable OR gates	Uses programmable AND and fixed OR gates
Provides high flexibility	Provides less flexibility
Less commonly used	More commonly used